

Multi-Stage Context Pruning for GUI Agents

Quoc Viet Nguyen

August 2026

Department of Artificial Intelligence and Robotics

The Graduate School

Sejong University

Multi-Stage Context Pruning for GUI Agents

Quoc Viet Nguyen

A thesis submitted to the Faculty of the Sejong University in
partial fulfillment of the requirements for the degree of Master
in Artificial Intelligence and Robotics

August 2026

Approved by Major Advisor
Professor Cheol Jeong

Multi-Stage Context Pruning for GUI Agents

by
Quoc Viet Nguyen

Approved -----
Professor Jin Young Lee, Chair of the committee

Approved -----
Professor Yukyung Choi, Member of the committee

Approved -----
Professor Cheol Jeong, Advisor

Abstract

Graphical user interface (GUI) agents follow user instructions by observing screenshots and selecting actions such as clicking, typing, or scrolling. They are commonly based on Transformer architectures that combine visual tokens from screenshots with textual tokens from instructions to predict the next action. The main efficiency challenge arises in multi-step tasks, where a goal requires a sequence of GUI operations instead of one isolated action. Such tasks require the model to remember prior screenshots, previous actions, producing long multimodal contexts and increasing inference cost during both prefilling and key-value (KV) cache decoding. This thesis addresses the efficiency problem with a multi-stage context pruning framework for GUI agent inference. The method first removes less important historical screenshot blocks before exact prefilling using a Taylor-based importance score. It then applies token-level pruning and hybrid decoding to reduce decode-time KV-cache cost while preserving action-relevant history. Experiments on AGENTNETBENCH and OSWORLD show that the framework maintains performance under compressed KV-cache budgets. Specifically, on AGENTNETBENCH, even at only 20% KV retention, it achieves 78.3% step success with Qwen3VL 8B and 57.2% with EvoCUA 8B, substantially preserving action quality under severe compression. As the budget increases to 40% and 60%, performance rises to 81.2% and 82.0% for Qwen3VL 8B, and to 63.6% and 64.8% for EvoCUA 8B, approaching or slightly exceeding full-cache performance at higher retention. These results indicate that GUI agent contexts should be compressed according to interaction structure rather than treated as flat token sequences.

Keywords: Context pruning, GUI agent, Key-value cache, Vision-language model.

Contents

Abstract	iii
List of Figures	ix
List of Tables	xiii
List of Acronyms	xv
1 Introduction	1
1.1 Background	1
1.2 Motivations	3
1.3 Contributions	7
1.4 Thesis Outline	8
2 Preliminaries	9
2.1 Notation	9
2.2 Transformer Model	17
2.3 Transformer Attention	19
2.4 Prefilling and Decoding in Autoregressive Inference	21
2.5 Key-Value Cache in Vision-Language Models	24
2.6 Eviction Loss	25
3 Related Work	27
3.1 Context Pruning in LLMs and VLMs	27
3.2 Context Pruning in GUI Agents	28

4	Methodology	31
4.1	Problem Formulation	31
4.2	Overview of the Proposed Framework	32
4.3	Stage 1: Step-Level Pruning	35
4.4	Stage 2: Token-Level Pruning and Hybrid Decoding	45
4.5	Complexity Analysis	49
5	Experimental Results and Discussion	53
5.1	Benchmarks	53
5.2	Metrics	54
5.3	Models	54
5.4	Baselines	55
5.5	Quantitative Results	56
5.6	Qualitative Results	62
5.7	Wall-Time and FLOPs Analysis	65
5.8	Hyperparameter Settings	69
5.9	Ablation Study	71
5.10	Performance of Different Step Pruning Approaches	72
5.11	Performance of Scoring Approximation and Pruning Scope	74
5.12	Performance with Varying Numbers of Screenshots	75
5.13	Discussion and Failure Cases	77
5.14	Visualization of Multi-Stage Pruning	80
6	Conclusion and Future Work	87
6.1	Conclusion	87
6.2	Future Work	89

Korean Abstract	97
Acknowledgement	99

List of Figures

1.1	Long-horizon GUI agent trajectory with instruction, historical screenshots, reasoning, actions, and the current screenshot.	2
1.2	Attention map over historical GUI steps, showing that the model extracts information from past screenshots selectively rather than uniformly. In some cases, an older screenshot such as step $t - 3$ can receive stronger attention than a more recent screenshot such as step $t - 2$	4
2.1	Transformer architecture [7] for unified text and visual token processing.	18
2.2	Attention computation from query-key scores to weighted value aggregation.	20
2.3	Prefilling stage in autoregressive inference. The model processes the full input context once, computes hidden states for all prompt tokens, and stores their key-value representations in the KV cache for later decoding.	22
2.4	Decoding stage in autoregressive inference. The model generates output tokens one at a time, reusing the KV cache from prefilling so each new query can attend to the previously processed context efficiently.	23

4.1	Comparison of full-cache inference, token-only pruning, and the proposed multi-stage pruning framework. Full-cache inference retains all historical screenshots and text tokens, token-only pruning compresses the KV cache after prefilling, and the proposed method first removes redundant historical screenshot blocks before token-level compression.	32
4.2	Stage 1 step-level pruning. Historical screenshot blocks are scored, low-importance visual blocks are removed before exact prefilling.	35
4.3	Theoretical FLOPs comparison between exact softmax scoring and the Taylor-approximated scoring.	38
4.4	Stage 2 token-level pruning and hybrid decoding. Important tokens in the reduced context are kept for exact softmax attention, while pruned tokens are summarized into compact statistics. . . .	46
5.1	Successful click prediction from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model selects the First class option with the correct action type and target location.	62
5.2	Successful text-entry prediction from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model writes Shanghai into the intended Leaving from field. .	63
5.3	Wall-time comparison across pruning methods and KV-cache budgets. Runtime is decomposed into prefilling, scoring, and decoding stages.	65

5.4	Theoretical FLOPs comparison across pruning methods and KV-cache budgets. The stacked bars separate prefilling, scoring, and decoding costs.	66
5.5	Grounding error from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the target is identified correctly, but the click location misses the one-way button.	77
5.6	Template and understanding error from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model generates an invalid key template and submits the query too early.	78
5.7	Stage 1 example 1. Red screenshots indicate low-importance screenshots that are pruned. This example shows that visually similar past screenshots such as t-4, t-3 and t-2 are pruned.	81
5.8	Stage 1 example 2. Red screenshots indicate low-importance screenshots that are pruned. In this example, t-5 and t-4 have the same low score, showing temporal redundancy, while the visually similar t-3 and t-2 are also pruned.	82
5.9	Stage 1 example 3. Red screenshots indicate low-importance screenshots that are pruned. This example shows that visually similar past screenshots such as t-4 and t-3 are pruned.	83
5.10	Stage 2 token pruning at the 20% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. This is the most aggressive setting, so only the most important tokens are directly retained.	84

5.11 Stage 2 token pruning at the 40% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. Compared with the 20% case, more tokens are directly preserved. 85

5.12 Stage 2 token pruning at the 60% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. This budget is more conservative, so a larger portion of the original context is retained. 86

List of Tables

2.1	Summary of notation used throughout the thesis.	9
4.1	Comparison between exact softmax scoring and the Taylor approximation written in direct and cumulative-statistics forms. The table shows how exponential softmax terms are replaced by first-order Taylor terms and how cumulative key, value, and key-value sums allow efficient block-level scoring.	40
4.2	Theoretical complexity comparison of full-cache inference, token pruning, and the proposed multi-stage context pruning pipeline.	50
5.1	Success rates on AGENTNETBENCH under different KV-cache budgets. The action categories are coordinate, content, and function, and the KV budget denotes the percentage of KV cache retained during inference. For each model, budget, and metric, boldface indicates the best score among SnapKV, GUI-KV, and Proposed, excluding Full Cache.	57
5.2	Success rates on OSWORLD under different KV-cache budgets. Tasks are grouped into OS, Office, Daily, Professional, and Workflow categories. The KV budget denotes the percentage of KV cache retained during inference. For each model, budget, and task category, boldface indicates the best score among SnapKV, GUI-KV, and Proposed, excluding Full Cache.	59

5.3	Hyperparameter sensitivity of Qwen3VL 8B on AGENTNETBENCH. We report the overall success rate under different KV-cache budgets, observation window sizes W , and normalized eviction thresholds γ . Boldface indicates the best score for each KV-cache budget.	69
5.4	Ablation study on AGENTNETBENCH with Qwen3VL 8B. The table reports overall step success rate under different KV-cache budgets. The symbols $\checkmark$ and $\times$ indicate whether Stage 1 step pruning and Stage 2 hybrid decoding are enabled. Boldface indicates the best score for each KV-cache budget.	71
5.5	Overall success rate of Qwen3VL 8B on AGENTNETBENCH under different Stage 1 pruning approaches. Random pruning removes historical screenshot blocks without an importance signal, far-to-near pruning removes older screenshots first, and Proposed uses the Taylor-approximated perturbation score for image-only pruning. Boldface indicates the best score for each KV-cache budget.	72
5.6	Performance of Stage 1 scoring approximation and pruning scope variants using Qwen3VL 8B on AGENTNETBENCH. The Approximation column identifies whether the Stage 1 score uses Taylor approximation, the Image and Text columns indicate whether screenshot tokens and associated text tokens are eligible for pruning. Boldface indicates the best score for each KV-cache budget.	74
5.7	Performance on AGENTNETBENCH with varying numbers of past steps. Boldface indicates the best score for each model.	75

List of Acronyms

CUA	Computer Use Agent
GUI	Graphical User Interface
KV	Key-Value
LLM	Large Language Model
VLM	Vision Language Model

Chapter 1

Introduction

1.1 Background

Graphical user interface (GUI) agents extend vision-language models (VLMs) from standard perception tasks on natural images to interactive GUI tasks on mobile phones and computers. Instead of answering a question from a single image, a GUI agent receives a user instruction, observes a sequence of screenshots and intermediate reasoning steps, and outputs actions such as clicking, typing, scrolling, or selecting interface elements. The decision at the current step therefore depends on a full interaction trajectory rather than an isolated prompt. As illustrated in Figure 1.1, the model must jointly maintain the task instruction, the visual history, and the intermediate reasoning and action records in order to predict the next GUI action correctly.

The input to a long-horizon GUI agent can be summarized as:

$$\text{Trajectory} = \{\text{system prompt, user task, past screenshots,} \\ \text{past reasoning and actions, current screenshot}\}. \quad (1.1)$$

This trajectory memory is important because it records previously seen GUI screenshot states, completed subgoals, and the textual reasoning that explains how the current state was reached. In many tasks, the next action of a GUI agent cannot be inferred from the current screenshot alone. The model must

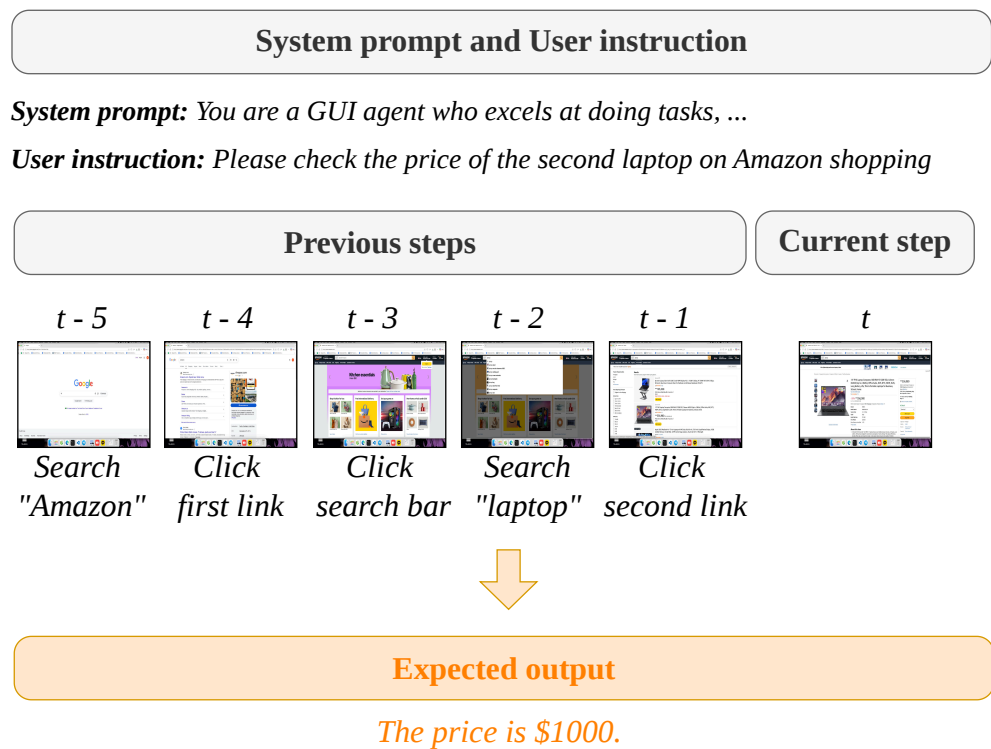


FIGURE 1.1: Long-horizon GUI agent trajectory with instruction, historical screenshots, reasoning, actions, and the current screenshot.

1.2. Motivations

remember what it has already tried, what information has been gathered inside the trajectory, and which interface elements were relevant several steps earlier.

Most recent GUI agents [1–6] are implemented with transformer-based [7] VLMs. Their inference usually has two stages: prefilling and decoding. Prefilling is the first pass over the input. In this stage, the model reads the whole multi-modal context, including the instruction, previous screenshots, previous actions, and the current screenshot. It computes attention over these input tokens and stores their key-value (KV) representations in the KV cache. Decoding happens after this first pass. In this stage, the model produces the action response one token at a time. At each step, it uses the current token as a query and looks up the stored KV cache to reuse the previous context instead of processing the whole trajectory again. This design is effective for GUI agents, but it makes long GUI trajectories computationally expensive and time-consuming because each screenshot contributes thousands of visual tokens, and a larger KV cache increases the memory access and attention cost during decoding.

1.2 Motivations

The efficiency problem in GUI agents comes from the fact that most recent systems are built on Transformer-based VLMs. Although these models are powerful, they become expensive when each action prediction have to condition on screenshots, instructions, reasoning, and previous actions text. This issue is especially important for GUI agents because they operate through sequential interactions. The agent observes the current screen, predicts the next action, executes it, and repeats this process until the task is completed. A moderate delay in the inference

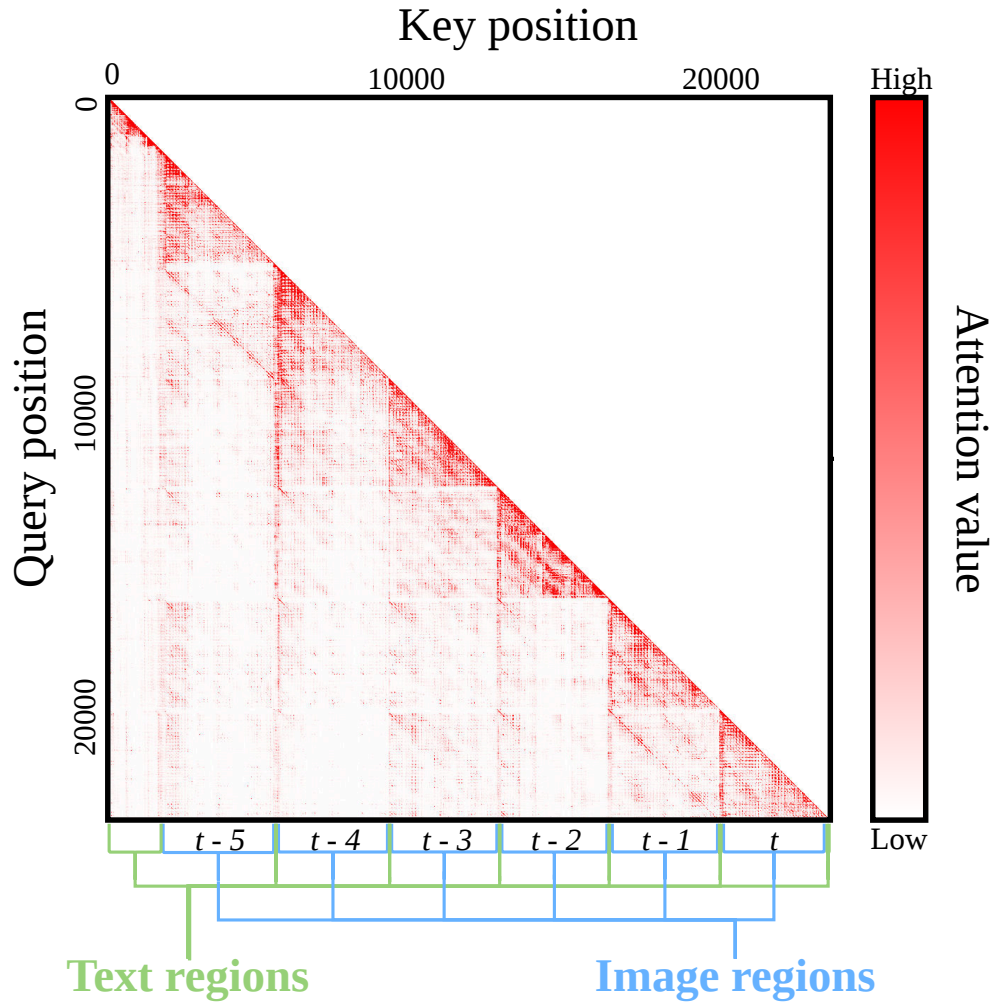


FIGURE 1.2: Attention map over historical GUI steps, showing that the model extracts information from past screenshots selectively rather than uniformly. In some cases, an older screenshot such as step $t - 3$ can receive stronger attention than a more recent screenshot such as step $t - 2$.

1.2. Motivations

step can accumulate across many actions, making the agent less practical for real-world use.

The main motivation of this thesis is that long-context GUI agent inference creates a severe KV cache bottleneck. As the number of historical steps increases, the sequence length grows rapidly. Exact self-attention during prefilling scales quadratically with sequence length, while the KV cache memory footprint grows linearly with the number of retained tokens. This cost is especially problematic for GUI trajectories because screenshots often dominate the token count.

However, GUI agent trajectories often contain strong structured redundancy. First, temporal redundancy appears across GUI agent screenshots. Throughout the trajectory, screenshots may differ only due to small scrolling motions, repeated layouts, menu toggles, or other minor interface updates. Some older screenshots become obsolete after the agent moves to a new UI state or completes a previous subgoal. Second, spatial redundancy also appears within each screenshot. Even when a screenshot remains useful, many visual tokens correspond to background regions, repeated icons, whitespace, or other local details irrelevant for the next decision.

Another important observation is that image tokens and text tokens do not play the same roles in GUI agent trajectories. Past textual reasoning and action records form a lightweight summary of execution history, while screenshot tokens dominate context length. Removing both modalities together may cause the model to lose visual evidence and continuity of the interaction process. This suggests a safer compression strategy is to prune less informative historical screenshot content while preserving textual action history as much as possible.

This intuition is also reflected in the attention pattern shown in Figure 1.2. The attention is not distributed uniformly across the past screenshots. Instead, the

model selectively extracts information from particular historical states according to their relevance to the current prediction. The nearest past step $t - 1$ receives dense attention, but the older step $t - 3$ can receive stronger attention than the intermediate step $t - 2$. This pattern shows that useful visual evidence is not necessarily concentrated in the most recent history. In some cases, the model needs to look back to an older GUI state to verify previous information or recover context needed for the next action. This observation supports a selective step-pruning strategy: weakly attended historical screenshots can be removed, but older screenshots should not be discarded solely because they are farther from the current step.

These observations motivate a multi-stage compression strategy. In the first stage, the method removes low-contribution historical screenshots before running the expensive exact softmax prefill on the remaining context. In the second stage, the sequence is compressed further at the token level during decoding. The first stage mainly deals with temporal redundancy across GUI agent interaction steps, while the second stage handles spatial and token-level redundancy within the reduced context.

The key technical challenge is estimating the importance of each historical step efficiently. The most intuitive approach is to check how much the output changes when a step is removed: if removing it causes a large shift in the attention output, the step should be kept; if the effect is small, the step can safely be pruned. However, computing this exact softmax-based perturbation for every step is too costly for long sequences. This thesis therefore uses a first-order Taylor approximation as a practical, linear-time alternative for scoring each step.

1.3 Contributions

The contributions of this thesis can be summarized as follows.

- A first-stage step-pruning method is proposed for long-horizon GUI agents. It scores historical screenshot blocks by estimating how much the attention output would change if each block were removed, reducing the number of tokens before the expensive full softmax prefill.
- A practical Taylor approximation is derived for this scoring process. By replacing the exact exponential weights with a first-order approximation and using cumulative statistics, the method keeps the structure of the exact pruning objective intact while avoiding the repeated quadratic-cost evaluations that exact scoring would require.
- After step reduction, a second-stage token-pruning and hybrid decoding scheme is introduced. Important tokens are handled with exact softmax attention, while less important ones are compressed into a small fixed-size summary state that feeds back in through a lightweight correction term during decoding.
- The overall framework is evaluated on two GUI agent settings using Qwen3VL 8B and EvoCUA 8B. The results show that the proposed multi-stage design is especially effective under tight compression budgets, and that step-aware pruning is a better fit for GUI trajectories than purely flat token pruning.

1.4 Thesis Outline

The remainder of this thesis is organized as follows.

1. Chapter 2 introduces the notation, transformer attention, autoregressive prefilling and decoding, the role of the KV cache in VLM, and the eviction-loss viewpoint used throughout the thesis.
2. Chapter 3 reviews previous KV cache compression methods for LLMs and VLMs and discusses the specific challenges of pruning context in GUI agents.
3. Chapter 4 presents the proposed multi-stage pruning framework, including exact step-importance formulation, Taylor-based step scoring, token-level keep and prune splitting, and hybrid decoding with a compressed prune set.
4. Chapter 5 describes the experimental setup and discusses the transferred benchmark results, ablations, efficiency trends, and failure cases.
5. Chapter 6 concludes the thesis and outlines several directions for future work.

Chapter 2

Preliminaries

2.1 Notation

This chapter introduces the basic concepts required for graphical user interface (GUI) agents, vision-language models (VLMs), and key-value (KV) cache pruning. Table 2.1 summarizes the main notation used throughout the thesis. Unless otherwise stated, lowercase letters denote scalars, lowercase bold or indexed symbols denote vectors, uppercase symbols denote matrices or aggregate terms, and calligraphic letters denote sets or structured blocks. The superscript $\top$ denotes transpose, $\|\cdot\|_2$ denotes the Euclidean norm, and $\|\cdot\|_F$ denotes the Frobenius norm for matrices.

TABLE 2.1: Summary of notation used throughout the thesis.

Notation	Description
X	Full GUI agent context or trajectory given to the model.
u	User instruction in the GUI agent trajectory.
$\mathcal{I}_s$	Visual tokens of the screenshot at interaction step s .

Notation	Description
$\mathcal{T}_s$	Textual reasoning and action tokens associated with step s .
S	Number of interaction steps or screenshots in the trajectory.
s, t	Interaction-step indices. The index t is used when referring to previous screenshots such as steps $t - 1$ and $t - 2$.
N	Total token length before pruning.
N'	Reduced token length after Stage 1 step pruning.
n	Query position in a causal attention layer.
i, j, m	Token or KV-cache position indices. In causal attention, visible positions satisfy $j \leq n$ or $m \leq n$.
l, r	Start and end token indices of a contiguous screenshot block.
q_n, q	Query vector at position n ; q is used for a generic decode-time query.
k_j, v_j	Key and value vectors at token position j .

1.4. Thesis Outline

Notation	Description
Q, K, V	Query, key, and value matrices used in compact matrix-form attention notation; each has shape $N \times d$.
$q_n^\top k_j$	Dot product between query q_n and key k_j ; $\top$ denotes transpose.
$a_{j,n}$	Exact softmax attention weight from query position n to key position j .
$o_n, o(q)$	Vector attention output at position n , or for a generic decode-time query q .
$O, O_{\text{softmax}}, O_{\text{Taylor}}$	Matrix-form attention outputs; O_{softmax} is produced by exact softmax attention and O_{Taylor} by Taylor-approximated attention. Matrix outputs have shape $N \times d$.
Y, z	Generic matrix and column vector used to define row-wise division, where $Y \in \mathbb{R}^{N \times d}$ and $z \in \mathbb{R}^{N \times 1}$.
$Y_{i,j}$	Element in row i and column j of matrix Y .
z_i	Element i of column vector z .
$(Y \oslash z)_{i,j}$	Element in row i and column j of the row-wise division result, equal to $Y_{i,j} / z_i$.

Notation	Description
$\oslash$	Row-wise division operator; $(Y \oslash z)_{i,j} = Y_{i,j}/z_i$.
$\mathbf{1}$	All-ones vector used in compact attention notation for row-wise summation, constant terms, and broadcasted matrix expressions.
$\mathbf{1}\mathbf{1}^\top$	All-ones matrix appearing in the Taylor approximation of the exponentiated query-key score matrix.
$K^\top V$	Compact key-value summary with shape $d \times d$ used in the Taylor matrix form.
$K^\top \mathbf{1}$	Compact key summary with shape $d \times 1$ used in the Taylor matrix form.
$N\mathbf{1}$	Constant-part normalization term with shape $N \times 1$ in the Taylor matrix form.
$\mathbf{1}(\mathbf{1}^\top V)$	Summed value information shared across queries with shape $N \times d$.
$\ell(\mathcal{B}, n)$	Generic eviction loss caused by removing candidate block $\mathcal{B}$ for query position n .
o_n^{full}	Attention output at query position n when the full prefix is retained.

1.4. Thesis Outline

Notation	Description
$o_n^{\text{pruned}}(\mathcal{B})$	Attention output at query position n after candidate block $\mathcal{B}$ is removed.
O^{full}	Matrix-form attention output before removing a candidate block.
$O^{\text{pruned}}(\mathcal{B})$	Matrix-form attention output after removing candidate block $\mathcal{B}$.
$\ell(\mathcal{B})$	Matrix-form eviction loss caused by removing candidate block $\mathcal{B}$, measured with the Frobenius norm.
$\mathcal{B}$	Candidate historical screenshot block considered for Stage 1 pruning.
$\mathcal{Q}$	Recent-query observation window used to score historical blocks.
$A_{\mathcal{B},n}^{\text{soft}}$	Exact softmax attention mass assigned to block $\mathcal{B}$ for query n .
$o_{\mathcal{B},n}^{\text{soft}}$	Exact attention output contributed by block $\mathcal{B}$ after normalizing within the block at position n .
$\ell_{\text{soft}}(\mathcal{B}, n)$	Exact softmax eviction loss caused by removing block $\mathcal{B}$ for query n .

Notation	Description
$\mathcal{S}_{\mathcal{B}}^{\text{soft}}$	Exact softmax importance score of candidate block $\mathcal{B}$ after normalizing and averaging over recent query positions.
$\tau_{j,n}$	First-order Taylor approximation of $\exp(q_n^\top k_j)$, defined as $1 + q_n^\top k_j$.
$\tilde{D}_n$	Taylor-approximated denominator of the full attention output at query n .
$\tilde{N}_n$	Taylor-approximated numerator of the full attention output at query n .
$\tilde{o}_n^{\text{full}}$	Taylor-approximated full attention output at query n .
$V_{\text{cs}}[n]$	Cumulative sum of value vectors up to position n .
$K_{\text{cs}}[n]$	Cumulative sum of key vectors up to position n .
$KV_{\text{cs}}[n]$	Cumulative sum of key-value outer products $k_j v_j^\top$ up to position n .
$V_{\mathcal{B}}, K_{\mathcal{B}}, KV_{\mathcal{B}}$	Block-level value, key, and key-value sums obtained by subtracting cumulative statistics at the block endpoints.
$ \mathcal{B} $	Number of tokens in candidate block $\mathcal{B}$.

1.4. Thesis Outline

Notation	Description
$\tilde{T}_{\mathcal{B},n}$	Taylor-approximated unnormalized mass of block $\mathcal{B}$ for query n .
$\tilde{N}_{\mathcal{B},n}$	Taylor-approximated numerator contributed by block $\mathcal{B}$ for query n .
$\tilde{A}_{\mathcal{B},n}$	Taylor-approximated normalized attention mass of block $\mathcal{B}$ for query n .
$\tilde{o}_{\mathcal{B},n}$	Taylor-approximated block output for block $\mathcal{B}$ for query n .
$\ell_{\text{Taylor}}(\mathcal{B}, n)$	Taylor eviction-loss surrogate for block $\mathcal{B}$ and query n .
ϵ	Small positive stability constant for normalization.
$\mathcal{S}_{\mathcal{B}}$	Final normalized importance score of candidate block $\mathcal{B}$.
γ	Stage 1 pruning threshold.
K_{total}	Overall token budget after the two-stage pruning pipeline; also the number of tokens retained for exact decode-time attention.
$\mathcal{K}$	Token-level keep set retained for exact attention during hybrid decoding.

Notation	Description
$\mathcal{P}$	Token-level prune set for later compression.
$K_{\mathcal{K}}, V_{\mathcal{K}}$	Key and value matrices corresponding to keep tokens in $\mathcal{K}$.
$o_{\mathcal{K}}$	Exact softmax attention output over keep tokens in $\mathcal{K}$.
$\mathcal{A}(\cdot)$	Attention operator that maps a query, keys, and values to an attention output.
A, C	Numerator and denominator contributions from keep tokens during hybrid decoding.
B, D	Numerator and denominator contributions from pruned tokens during hybrid decoding.
v_{sum}	Sum of value vectors over pruned tokens.
k_{sum}	Sum of key vectors over pruned tokens.
$S_{\mathcal{P}}$	Sum of key-value outer products over pruned tokens, $\sum_{j \in \mathcal{P}} k_j v_j^{\top}$.
$L_{\mathcal{P}}$	Number of pruned tokens, equal to $ \mathcal{P} $.
o	Final hybrid decode output after combining exact keep-token attention with the pruned-token correction term.

2.2. Transformer Model

Notation	Description
d	Per-head hidden dimension used in memory-complexity discussion.
$O(\cdot)$	Big-O asymptotic complexity notation.
W	Observation window size used in the hyperparameter study.
$\uparrow$	Indicates that a larger evaluation value is better.

In the GUI agent setting, one trajectory consists of a system prompt, a user instruction, a sequence of past screenshots, a sequence of past textual reasonings and actions, and a current screenshot. The goal of context pruning is to reduce the computational cost and time of processing this trajectory while preserving the information that is most relevant for predicting the next action.

2.2 Transformer Model

Transformer models [7–11] process a sequence of tokens by repeatedly applying normalization, self-attention and feed-forward transformations. In a large language model (LLM) or a vision-language model (VLM), text tokens are obtained from the tokenized system prompt, user instruction, and generated reasoning, while image tokens are obtained by projecting image patches or visual embeddings into the same hidden space as text tokens. The model then processes all tokens in a unified transformer stack. As shown in Figure 2.1, this unified design

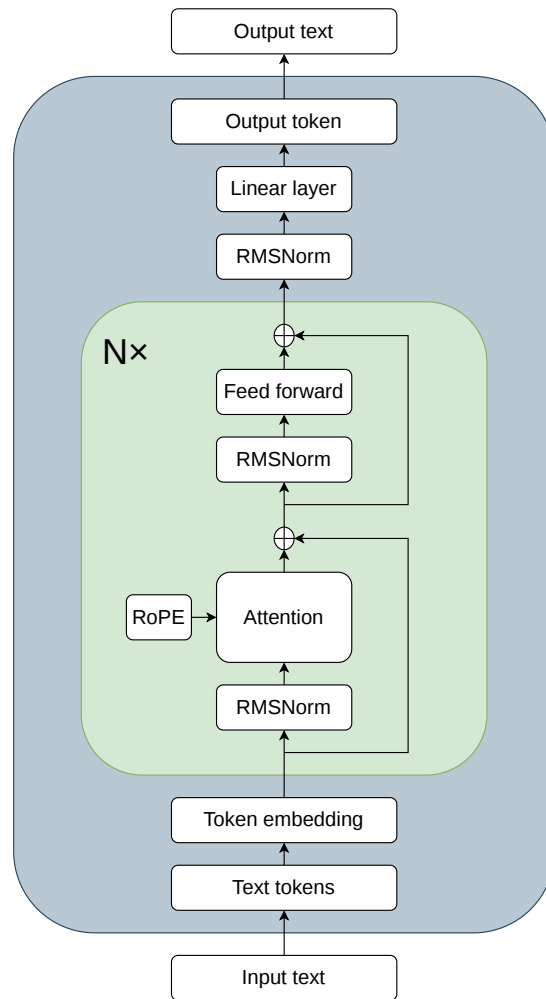


FIGURE 2.1: Transformer architecture [7] for unified text and visual token processing.

2.3. Transformer Attention

allows the same Transformer backbone to integrate multimodal inputs within one sequence-processing architecture, which is one of the main reasons why it is effective for GUI agents.

For GUI agents, this architecture is especially natural because a trajectory is inherently multimodal. Text tokens are used to describe the task and the executed actions, while visual tokens encode the content of each screenshot. The transformer layers allow the current query positions to gather information from both modalities. This helps the model use visual evidence to choose actions while still keeping track of what has happened across multiple steps.

However, this flexibility also creates a practical problem. As more screenshots and past actions are added, the input sequence becomes much longer and more expensive to process. The proposed compression framework therefore keeps the same transformer architecture, but reduces the effective context length while trying to preserve the most important historical information.

2.3 Transformer Attention

In a transformer layer, the attention output at query position n is a weighted average of value vectors from all visible positions:

$$o_n = \sum_{j \leq n} a_{j,n} v_j. \quad (2.1)$$

Here, o_n denotes the attention output vector at query position n , j indexes a visible previous or current token position, v_j denotes the value vector at position j , and $a_{j,n}$ denotes the attention weight assigned from query position n to value position j . The condition $j \leq n$ reflects causal attention, where a token can only attend to

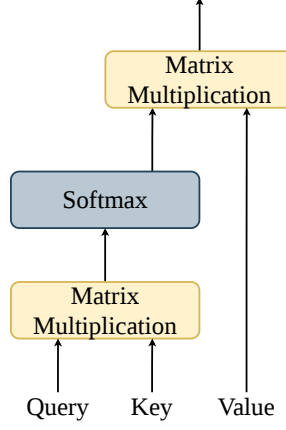


FIGURE 2.2: Attention computation from query-key scores to weighted value aggregation.

itself and earlier tokens. The attention weights are obtained by comparing the query vector q_n with all key vectors k_j through a scaled dot product and then normalizing the scores with softmax:

$$a_{j,n} = \frac{\exp(q_n^\top k_j)}{\sum_{m \leq n} \exp(q_n^\top k_m)}. \quad (2.2)$$

In this expression, q_n is the query vector at position n , k_j is the key vector at visible position j , and $q_n^\top k_j$ is their dot-product similarity score. The exponential function $\exp(\cdot)$ converts each similarity score into a positive unnormalized weight, while the denominator sums these unnormalized weights over all visible positions $m \leq n$ so that the normalized weights $a_{j,n}$ sum to one. The softmax converts raw similarity scores into a probability distribution. Tokens whose keys align strongly with the query receive higher weights and contribute more to the output. Tokens with small or irrelevant scores receive negligible weight. This mechanism is central to both the usefulness and the cost of long-context inference. It allows the model to retrieve relevant information from far in the past, but it

2.4. Prefilling and Decoding in Autoregressive Inference

also requires evaluating a large number of query-key pairs. As illustrated in Figure 2.2, attention can be understood as a three-stage process: score computation from query-key similarity, softmax normalization into weights, and weighted aggregation of value vectors into the final output.

In GUI trajectories, attention is not uniformly distributed across the history. Some past screenshots are nearly irrelevant, while a more distant screenshot can become important again when the model needs to verify an earlier interface state. This uneven importance distribution is one of the main reasons why attention-based pruning is a promising strategy.

2.4 Prefilling and Decoding in Autoregressive Inference

Autoregressive inference usually proceeds in two stages: prefilling and decoding. Figure 2.3 on the next page illustrates the prefilling stage. The full input prompt is processed at once, so the model computes query, key, and value representations for all prompt tokens in parallel. The first generated token is produced from this full-context computation, and the key-value pairs of the prompt tokens are stored in the KV cache for later reuse.

Figure 2.4 on the following page shows the decoding stage. After the first generated token is available, the model no longer recomputes the full prompt from scratch. Instead, it uses the current token as the new query and loads the previous keys and values from the cache. The model then produces the next token and appends the new key and value to the cache. This process repeats one token at a time, which makes decoding efficient compared with recomputing the whole history, but it also means that every decoding step depends on the size of the retained KV cache.

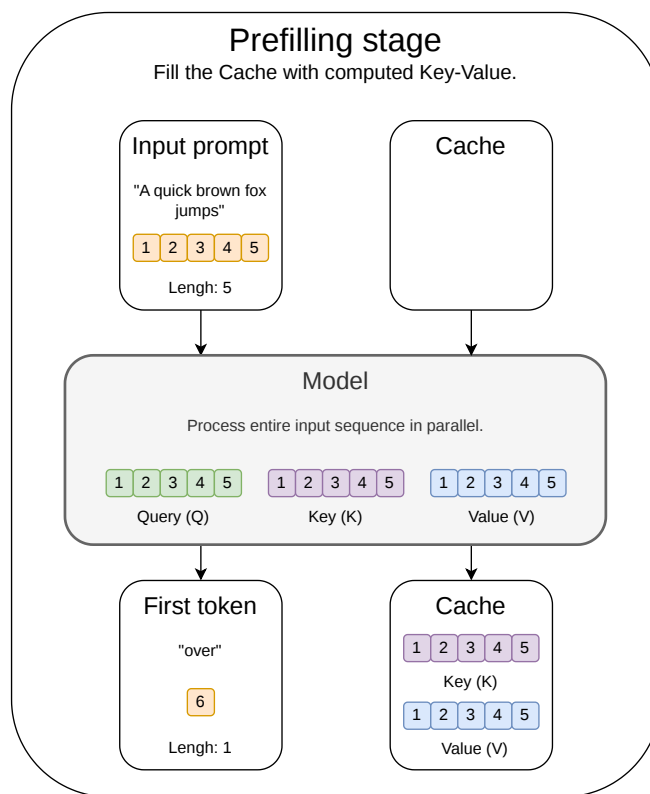


FIGURE 2.3: Prefilling stage in autoregressive inference. The model processes the full input context once, computes hidden states for all prompt tokens, and stores their key-value representations in the KV cache for later decoding.

2.4. Prefilling and Decoding in Autoregressive Inference

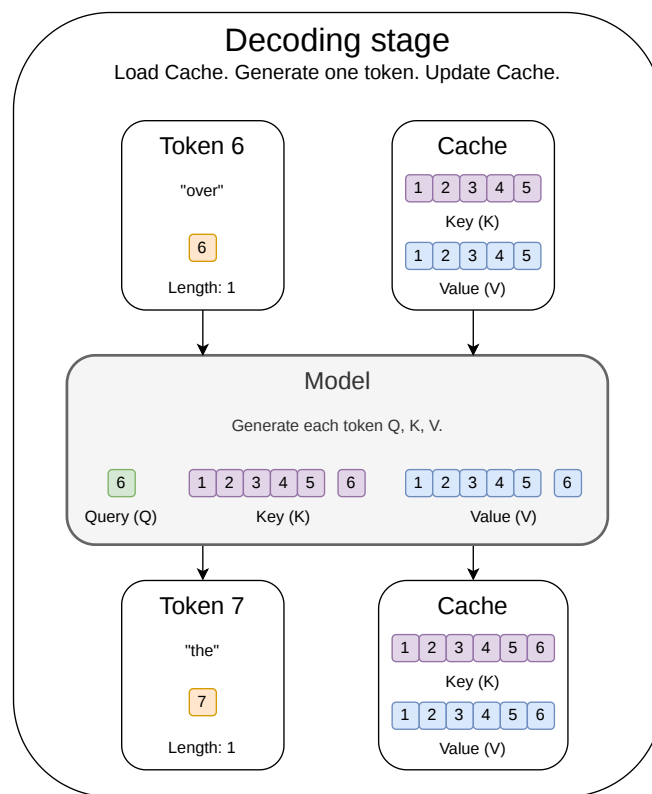


FIGURE 2.4: Decoding stage in autoregressive inference. The model generates output tokens one at a time, reusing the KV cache from prefilling so each new query can attend to the previously processed context efficiently.

This decomposition is essential for efficient generation, but its cost depends strongly on context length. Prefilling is expensive because it applies exact attention to the whole input sequence. Decoding is cheaper per token, but its memory use and attention lookup cost still increase as the retained KV cache becomes larger. For GUI agents with multiple screenshots, both stages become bottlenecks: the prefill sequence is long because screenshots add many visual tokens, and the retained cache is heavy because the historical visual context remains available during decoding.

The proposed method in this thesis explicitly targets both stages. Stage 1 reduces the sequence before exact prefill, lowering the upfront cost shown in Figure 2.3. Stage 2 compresses the retained cache for subsequent decoding, reducing the repeated cache-access cost shown in Figure 2.4.

2.5 Key-Value Cache in Vision-Language Models

The KV cache stores the key and value tensors produced by the transformer layers for all previously processed tokens. In language-only models, the cache already grows substantially with long prompts. In vision-language models, the problem is more severe because each screenshot contributes a large block of visual tokens. A trajectory with several past screenshots can therefore reach tens of thousands of tokens even before the current decoding step begins.

This growth creates two related problems. First, the prefill computation over the full context becomes expensive because attention complexity grows quadratically with sequence length. Second, the decode-time KV cache becomes large, increasing memory use and the cost of every subsequent attention lookup. In GUI agents, the challenge is compounded by the fact that not all retained

tokens are equally useful. Some screenshot regions are redundant, some whole screenshots are obsolete, and some historical text is much more informative than the surrounding visual content.

These observations suggest that context compression should not be flat. Instead, it should exploit the structure of GUI agent trajectories: historical-step redundancy and token-level redundancy within each retained step.

2.6 Eviction Loss

To decide whether a past context block can be removed, this thesis adopts an output-perturbation viewpoint called Eviction Loss [12]. Let o_n^{full} denote the attention output at query position n when the full prefix is used, and let $o_n^{\text{pruned}}(\mathcal{B})$ denote the output after removing a candidate historical block $\mathcal{B}$. The Eviction loss is defined as

$$\ell(\mathcal{B}, n) = \left\| o_n^{\text{full}} - o_n^{\text{pruned}}(\mathcal{B}) \right\|_2. \quad (2.3)$$

Here, $\ell(\mathcal{B}, n)$ denotes the eviction loss for candidate block $\mathcal{B}$ at query position n , and $\| \cdot \|_2$ denotes the Euclidean norm of the difference between the full and pruned attention outputs. This quantity measures how much the model output changes when the candidate block is removed. A large value means that the removed block strongly affects the current attention output and should likely be kept. A small value means that the block is comparatively unimportant and is a good pruning candidate.

The exact Eviction Loss provides a clean conceptual target, but computing it directly for every step is expensive in long-context transformer inference. The central idea of the method chapter is therefore to keep this loss as the guiding

objective while deriving cheaper approximations that are practical for GUI agent trajectories.

Chapter 3

Related Work

3.1 Context Pruning in LLMs and VLMs

Many studies improve long-context efficiency by reducing the number of tokens in the KV cache and can be grouped into several related directions. Early methods such as StreamingLLM [13] retain attention sinks together with a recent sliding window, while Scissorhands [14] maintains a fixed budget based on the persistence of token importance over time. Subsequent methods move toward more explicit importance estimation. Keyformer [15] retains key tokens that dominate attention mass, SnapKV [16] selects cache entries from an observation window near the end of prefilling, and Quest [17] further makes selection query-aware during long-context inference. These methods share the core idea that only a small subset of tokens needs to remain active for accurate generation.

Subsequent studies have refined this line of research by considering how compression should adapt to model depth or modality. PyramidKV [18] observes that attention is broad in lower layers while more concentrated in higher layers, and uses this pyramidal pattern to allocate larger cache budgets to lower layers and smaller budgets to upper layers. Ada-KV [12] allocates cache budgets adaptively across heads. Relatedly, HeadKV [19] performs head-level allocation using retrieval and reasoning cues, while RazorAttention [20] separates retrieval heads from local heads and preserves full context only where necessary. Compared

with earlier fixed-budget approaches, these methods treat cache compression as a structured allocation problem over heads or layers rather than a flat top- k selection problem.

Another line of work reduces computation before cache construction by compressing the prompt. LLMingua [21], LongLLMingua [22], and LLMingua-2 [23] develop increasingly stronger multi-stage and distilled prompt compression strategies for long-context settings. Although prompt compression operates at a different stage from KV eviction, it addresses the same underlying problem: large portions of the input context are often redundant for the final prediction.

Extending token pruning to vision-language and multimodal models, FastV [24], SparseVLM [25], and VisionZip [26] prune or compress visual tokens by exploiting redundancy across layers or attention patterns. For long-context multimodal inference, LOOK-M [27] and MadaKV [28] show that textual and visual tokens should not share the same compression policy: both methods introduce modality-aware cache eviction, with stronger compression applied to image-heavy cache regions and compensatory mechanisms to reduce information loss.

3.2 Context Pruning in GUI Agents

For agents, the efficiency problem involves not only long-context inference for a single prompt, but also the continual growth of screenshots, observations, action histories, and reasoning traces over long horizons. Early studies therefore focus on GUI-aware screenshot token reduction that requires training. ShowUI [5] introduces UI-guided visual token selection by explicitly modeling structural redundancy in interface layouts, showing that GUI screenshots should not be

3.2. Context Pruning in GUI Agents

treated like natural images. FocusUI [29] sharpens this perspective by showing that token dropping can break positional continuity and harm grounding accuracy; it therefore combines query-guided token selection with a position-preserving padding mechanism tailored to UI localization. These studies are closest to training-based token pruning for GUI perception.

For training-free cache compression in GUI trajectories, GUI-KV [30] directly targets long screenshot histories in GUI agents and shows that GUI workloads exhibit distinctive spatial and temporal redundancy. Based on this observation, GUI-KV proposes uniform layer budgets, spatial saliency guidance, and temporal redundancy scoring over past screenshots.

Chapter 4

Methodology

4.1 Problem Formulation

Consider a GUI agent trajectory composed of a user instruction, a sequence of historical screenshots, a sequence of intermediate textual reasonings and actions, and the current screenshot. Let the overall context be written as

$$X = \{u, (\mathcal{I}_1, \mathcal{T}_1), (\mathcal{I}_2, \mathcal{T}_2), \dots, (\mathcal{I}_{S-1}, \mathcal{T}_{S-1}), \mathcal{I}_S\}, \quad (4.1)$$

where u is the user instruction, $\mathcal{I}_s$ denotes the visual tokens of the screenshot at step s , and $\mathcal{T}_s$ denotes the associated reasoning and action tokens produced after that step. Here, S is the total number of GUI steps, and the final screenshot $\mathcal{I}_S$ denotes the current visual observation used to predict the next action. In long-horizon tasks, the total token length of X becomes large because each historical screenshot can contribute many visual tokens.

The goal of this thesis is to reduce both the prefill cost and the decode-time KV-cache cost while preserving the information needed for the next action prediction. The key observation is that the heaviest part of the context is usually the historical visual content. A screenshot contributes more tokens than the short reasoning and action text associated with the step, while the text often provides a compact record of task progress and executed actions. Therefore, Stage 1 is designed to target image content only. Historical screenshot blocks are eligible

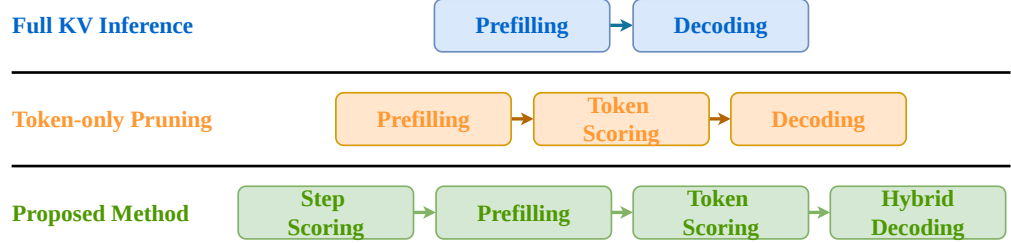


FIGURE 4.1: Comparison of full-cache inference, token-only pruning, and the proposed multi-stage pruning framework. Full-cache inference retains all historical screenshots and text tokens, token-only pruning compresses the KV cache after prefilling, and the proposed method first removes redundant historical screenshot blocks before token-level compression.

for removal, but textual reasoning and action tokens are preserved as much as possible to maintain the continuity of the interaction trajectory. This design is also consistent with modality-aware VLM compression studies such as LOOK-M [27] and MadaKV [28], which show that visual and textual tokens can have different importance patterns and should not necessarily share one uniform pruning policy.

Let K_{total} denote the overall token budget after both stages. The desired system should satisfy two competing requirements: it should reduce the cost of long-context inference, and it should keep the output perturbation caused by pruning sufficiently small. The method developed in this chapter achieves this by combining Stage 1 step pruning before exact prefill with Stage 2 token pruning after step reduction.

4.2 Overview of the Proposed Framework

The proposed framework follows a multi-stage pipeline designed for the structure of GUI agent trajectories. Figure 4.1 compares three inference strategies: full

4.2. Overview of the Proposed Framework

KV-cache inference, token-only pruning, and the proposed method. In full-cache inference, every historical screenshot and text token is retained, so the model pays the full prefill cost and carries the full KV cache into decoding. In token-only pruning, compression is applied after the long context has already been processed, which reduces decode-time cache size but does not remove the expensive prefill over redundant historical screenshots. In contrast, the proposed method first removes less informative historical screenshot blocks before exact prefilling and then applies token-level compression to the reduced context during decoding. Therefore, the framework targets both major costs of long-horizon GUI agents: the quadratic cost of prefilling and the repeated cache-access cost during decoding.

The overall pipeline consists of three connected components. First, the trajectory is decomposed into structured blocks, where each historical step contains screenshot tokens and associated textual reasoning or action tokens. Because screenshots usually dominate the token count, Stage 1 focuses on pruning historical visual blocks while preserving textual action history whenever possible. Second, the retained trajectory is prefilled with exact softmax attention, so the model still builds high-quality hidden states and KV pairs for the context that remains after step pruning. Third, the retained KV cache is further compressed at the token level, and decoding is performed with exact attention over important tokens plus a lightweight correction term for the pruned tokens. Unlike Stage 1, Stage 2 is not restricted to screenshot blocks or visual token only. It operates over the reduced token sequence, therefore, redundant text tokens can also be compressed if the token-level score of a text token is low.

Stage 1: Step-Level Pruning. Before running the expensive exact softmax prefill on the final retained context, the method estimates the importance of each historical screenshot block. The importance score is based on how much the attention output would change if that block were removed. Since exact softmax-based scoring is too expensive, a first-order Taylor approximation is used to produce a practical surrogate. Low-scoring historical screenshot blocks are pruned first, which shortens the sequence from the full length N to a reduced length N' . After step pruning, the model performs the exact softmax prefill on the shorter sequence. This is important because the final retained context should still be processed exactly during the prefill stage. The reduction from N to N' directly lowers the quadratic cost of this stage.

Stage 2: Token-Level Pruning and Hybrid Decoding. Inside the reduced context, the method performs token-level compression. The remaining tokens are split into a keep set and a prune set. Keep tokens are attended with exact softmax during decoding, while pruned tokens are summarized into a fixed-size compressed state. During each decode step, the model computes exact attention over the keep set and then adds a lightweight correction term that approximates the contribution of the pruned tokens. This allows the decoder to keep most of the benefit of a larger context without storing or attending to every low-importance token individually.

This design matches the hierarchical redundancy of GUI trajectories. Entire historical screenshots may be redundant at a step level, while individual tokens inside the remaining screenshots may still be redundant at a token level. By aligning the compression stages with this hierarchy, the proposed framework avoids treating the GUI trajectory as a flat sequence and instead removes redundancy at

4.3. Stage 1: Step-Level Pruning

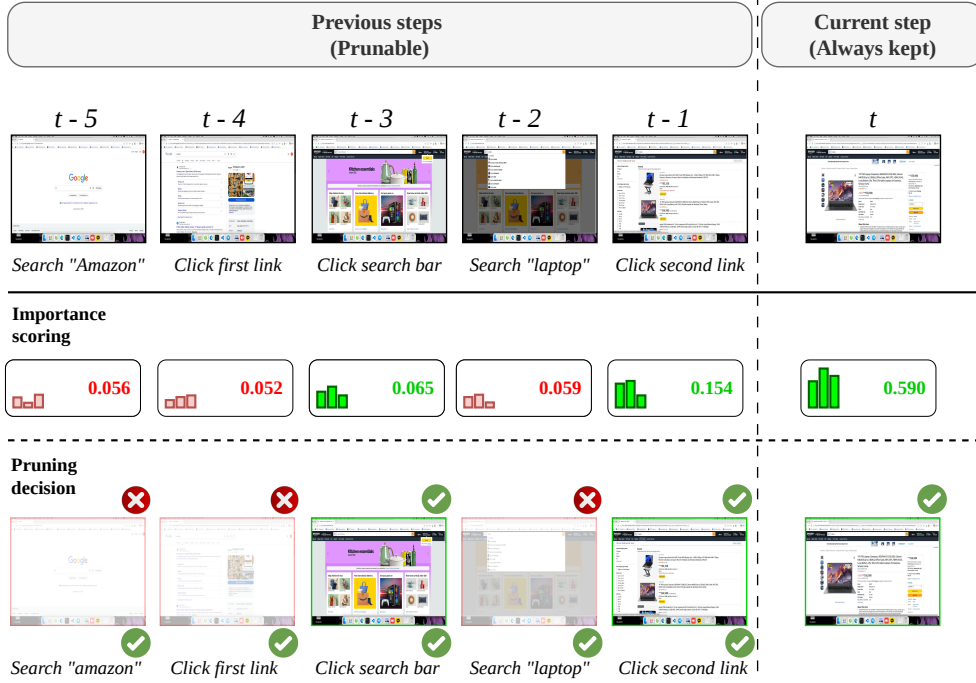


FIGURE 4.2: Stage 1 step-level pruning. Historical screenshot blocks are scored, low-importance visual blocks are removed before exact prefilling.

the level where it naturally appears.

4.3 Stage 1: Step-Level Pruning

Figure 4.2 illustrates the first stage of the proposed framework. The historical trajectory contains several past GUI states, such as steps $s-5$ through $s-1$, while the current step s is preserved because it provides the immediate visual evidence for the next action. Stage 1 assigns an importance score to each historical screenshot block and prunes low-scoring blocks before exact prefilling. This design removes redundant visual history early, so the expensive prefill computation is applied only to the retained trajectory.

The natural importance criterion for a historical block is the change in attention output caused by removing it. Under exact causal softmax attention, let q_n be the query vector at position n , and let k_j and v_j be the key and value vectors at previous cache position j . The index n denotes the query position whose attention output is being evaluated, while j indexes tokens in the causal prefix. The weight assigned by query position n to key position j is

$$a_{j,n} = \frac{\exp(q_n^\top k_j)}{\sum_{m=0}^n \exp(q_n^\top k_m)}, \quad (4.2)$$

and the corresponding attention output is

$$o_n^{\text{full}} = \sum_{j=0}^n a_{j,n} v_j. \quad (4.3)$$

Let $\mathcal{B}$ denote one candidate historical screenshot block, consisting of the visual token positions associated with one past GUI screenshot. For this candidate block, define its total attention mass and normalized block output as

$$A_{\mathcal{B},n}^{\text{soft}} = \sum_{j \in \mathcal{B}} a_{j,n}, \quad o_{\mathcal{B},n}^{\text{soft}} = \frac{\sum_{j \in \mathcal{B}} a_{j,n} v_j}{A_{\mathcal{B},n}^{\text{soft}}}. \quad (4.4)$$

After removing $\mathcal{B}$, the remaining weights are renormalized, and the resulting output can be written as

$$o_n^{\text{pruned}}(\mathcal{B}) = \frac{o_n^{\text{full}} - A_{\mathcal{B},n}^{\text{soft}} o_{\mathcal{B},n}^{\text{soft}}}{1 - A_{\mathcal{B},n}^{\text{soft}}}. \quad (4.5)$$

Therefore, the exact softmax eviction loss becomes

$$\ell_{\text{soft}}(\mathcal{B}, n) = \left\| o_n^{\text{full}} - o_n^{\text{pruned}}(\mathcal{B}) \right\|_2 = \frac{A_{\mathcal{B},n}^{\text{soft}}}{1 - A_{\mathcal{B},n}^{\text{soft}}} \left\| o_{\mathcal{B},n}^{\text{soft}} - o_n^{\text{full}} \right\|_2. \quad (4.6)$$

4.3. Stage 1: Step-Level Pruning

This equation gives an intuitive interpretation. A block is important when it receives non-negligible attention mass and when the average value direction inside that block differs from the overall output direction. Because attention-output magnitudes can vary across query positions, the final exact softmax block score averages the eviction loss after normalizing it by the full-output magnitude:

$$\mathcal{S}_B^{\text{soft}} = \frac{1}{|\mathcal{Q}|} \sum_{n \in \mathcal{Q}} \frac{\ell_{\text{soft}}(\mathcal{B}, n)}{\|o_n^{\text{full}}\|_2 + \epsilon}. \quad (4.7)$$

Here, $\mathcal{S}_B^{\text{soft}}$ denotes the exact softmax importance score of candidate block $\mathcal{B}$, $|\mathcal{Q}|$ denotes the number of recent query positions used for scoring, and $\epsilon > 0$ is a small stability constant. A smaller value of $\mathcal{S}_B^{\text{soft}}$ means that removing block $\mathcal{B}$ causes a smaller relative perturbation to the attention output, so the block is a stronger pruning candidate.

Therefore, computing the exact softmax score for one candidate block follows four steps. First, compute the exact softmax weights $a_{j,n}$ over the full causal prefix for each recent query $n \in \mathcal{Q}$. Second, sum the weights inside the candidate block to obtain $A_{B,n}^{\text{soft}}$ and compute the block output $o_{B,n}^{\text{soft}}$. Third, use these quantities to compute the eviction loss $\ell_{\text{soft}}(\mathcal{B}, n)$. Finally, normalize each loss by $\|o_n^{\text{full}}\|_2 + \epsilon$ and average the normalized losses over $\mathcal{Q}$ to obtain $\mathcal{S}_B^{\text{soft}}$.

However, evaluating this exact softmax score for every candidate block is expensive because the softmax attention weights are globally normalized. For each query position n , the denominator $\sum_{m=0}^n \exp(q_n^\top k_m)$ depends on all previous keys in the prefix, not only on the candidate block. Therefore, computing $A_{B,n}^{\text{soft}}$, $o_{B,n}^{\text{soft}}$, o_n^{full} , and $\mathcal{S}_B^{\text{soft}}$ requires access to the full query-key score vector and the corresponding value-weighted sum. If there are many historical screenshot

blocks, exact scoring must repeat this procedure for many candidates and potentially for many query positions. This introduces a large number of dot products, exponentials, reductions, and normalizations over the full prefix. In a long GUI trajectory, where screenshots dominate the token count, this scoring cost can approach the cost of an additional attention pass over the original long context. As a result, exact softmax scoring would reduce the benefit of pruning before prefill: the method would spend substantial computation measuring which blocks to remove before it can actually shorten the sequence.

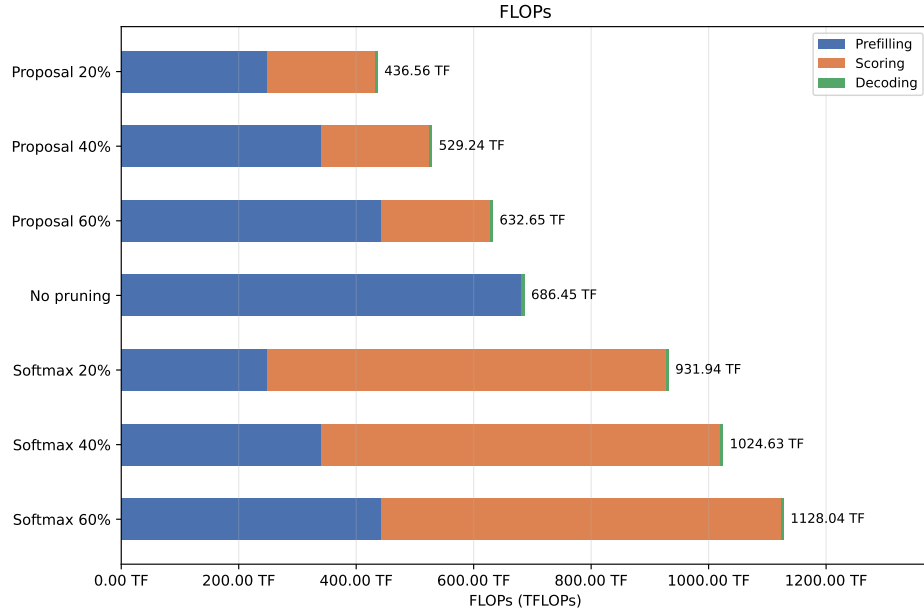


FIGURE 4.3: Theoretical FLOPs comparison between exact softmax scoring and the Taylor-approximated scoring.

Figure 4.3 illustrates the theoretical FLOPs comparison between the proposed Taylor-approximated scoring and an exact softmax scoring variant. With Taylor scoring, the proposed method requires 436.56, 529.24, and 632.65 TFLOPs at the 20%, 40%, and 60% budgets, respectively. Replacing the scoring stage with exact softmax scoring increases the corresponding costs to 931.94, 1024.63, and

4.3. Stage 1: Step-Level Pruning

1128.04 TFLOPs. The no-pruning setting requires 686.45 TFLOPs. These results show that exact softmax scoring can make the pruning procedure more expensive than full-cache inference because it repeatedly evaluates globally normalized attention for candidate screenshot blocks before the sequence is shortened. In contrast, the Taylor approximation enables early step-level pruning with much lower scoring cost by using cumulative key, value, and key-value statistics. In the Taylor approximation, $K_{cs}[n]$, $V_{cs}[n]$, and $KV_{cs}[n]$ denote cumulative sums of keys, values, and key-value outer products up to position n , respectively. If a block $\mathcal{B}$ spans token positions l to r , its block-level statistics can be obtained by subtracting cumulative sums at r and $l - 1$.

To obtain a more efficient scoring rule, the exponential term is approximated with its first-order Taylor expansion:

$$\exp(q_n^\top k_j) \approx 1 + q_n^\top k_j =: \tau_{j,n}. \quad (4.8)$$

Using this approximation, define

$$\tilde{D}_n = \sum_{j=0}^n \tau_{j,n}, \quad \tilde{N}_n = \sum_{j=0}^n \tau_{j,n} v_j, \quad \tilde{o}_n^{\text{full}} = \frac{\tilde{N}_n}{\tilde{D}_n}. \quad (4.9)$$

Since $\tau_{j,n} = 1 + q_n^\top k_j$, both the approximate denominator and numerator can be separated into a query-independent cumulative term and a query-dependent inner-product term. For the denominator,

$$\tilde{D}_n = \sum_{j=0}^n (1 + q_n^\top k_j) = (n+1) + q_n^\top \sum_{j=0}^n k_j. \quad (4.10)$$

Component	Exact softmax scoring	Taylor approximation
Token weight term	$\exp(q_n^\top k_j)$	$\tau_{j,n} = 1 + q_n^\top k_j$
Full normalization term	$D_n = \sum_{j=0}^n \exp(q_n^\top k_j)$	$\tilde{D}_n = \sum_{j=0}^n \tau_{j,n} = \sum_{j=0}^n (1 + q_n^\top k_j)$ $= (n+1) + q_n^\top K_{cs}[n]$
Full numerator	$N_n = \sum_{j=0}^n \exp(q_n^\top k_j) v_j$	$\tilde{N}_n = \sum_{j=0}^n \tau_{j,n} v_j = \sum_{j=0}^n (1 + q_n^\top k_j) v_j$ $= V_{cs}[n] + q_n^\top KV_{cs}[n]$
Full attention output	$o_n^{\text{full}} = N_n / D_n$	$\tilde{o}_n^{\text{full}} = \frac{\tilde{N}_n}{\tilde{D}_n} = \frac{\sum_{j=0}^n \tau_{j,n} v_j}{\sum_{j=0}^n \tau_{j,n}}$ $= \frac{V_{cs}[n] + q_n^\top KV_{cs}[n]}{(n+1) + q_n^\top K_{cs}[n]}$
Block statistics	$A_{\mathcal{B},n}^{\text{soft}} = \sum_{j \in \mathcal{B}} a_{j,n}$	$K_{\mathcal{B}} = \sum_{j \in \mathcal{B}} k_j = K_{cs}[r] - K_{cs}[l-1]$ $V_{\mathcal{B}} = \sum_{j \in \mathcal{B}} v_j = V_{cs}[r] - V_{cs}[l-1]$ $KV_{\mathcal{B}} = \sum_{j \in \mathcal{B}} k_j v_j^\top = KV_{cs}[r] - KV_{cs}[l-1]$
Block mass	$A_{\mathcal{B},n}^{\text{soft}} = \sum_{j \in \mathcal{B}} a_{j,n}$	$\tilde{T}_{\mathcal{B},n} = \sum_{j \in \mathcal{B}} \tau_{j,n} = \sum_{j \in \mathcal{B}} (1 + q_n^\top k_j)$ $= \mathcal{B} + q_n^\top K_{\mathcal{B}}$ $\tilde{A}_{\mathcal{B},n} = \frac{\tilde{T}_{\mathcal{B},n}}{\tilde{D}_n}$
Block output	$o_{\mathcal{B},n}^{\text{soft}} = \frac{\sum_{j \in \mathcal{B}} a_{j,n} v_j}{A_{\mathcal{B},n}^{\text{soft}}}$	$\tilde{N}_{\mathcal{B},n} = \sum_{j \in \mathcal{B}} \tau_{j,n} v_j = \sum_{j \in \mathcal{B}} (1 + q_n^\top k_j) v_j$ $= V_{\mathcal{B}} + q_n^\top KV_{\mathcal{B}}$ $\tilde{o}_{\mathcal{B},n} = \frac{\tilde{N}_{\mathcal{B},n}}{\tilde{T}_{\mathcal{B},n}}$
Eviction loss	$\ell_{\text{soft}} = \frac{A_{\mathcal{B},n}^{\text{soft}}}{1 - A_{\mathcal{B},n}^{\text{soft}}} \ o_{\mathcal{B},n}^{\text{soft}} - o_n^{\text{full}}\ _2$	$\ell_{\text{Taylor}} = \frac{\tilde{A}_{\mathcal{B},n}}{1 - \tilde{A}_{\mathcal{B},n}} \ \tilde{o}_{\mathcal{B},n} - \tilde{o}_n^{\text{full}}\ _2$
Final block score	$\mathcal{S}_{\mathcal{B}}^{\text{soft}} = \frac{1}{ \mathcal{Q} } \sum_{t \in \mathcal{Q}} \frac{\ell_{\text{soft}}(\mathcal{B}, n)}{\ o_n^{\text{full}}\ _2 + \epsilon}$	$\mathcal{S}_{\mathcal{B}} = \frac{1}{ \mathcal{Q} } \sum_{t \in \mathcal{Q}} \frac{\ell_{\text{Taylor}}(\mathcal{B}, n)}{\ \tilde{o}_n^{\text{full}}\ _2 + \epsilon}$

TABLE 4.1: Comparison between exact softmax scoring and the Taylor approximation written in direct and cumulative-statistics forms. The table shows how exponential softmax terms are replaced by first-order Taylor terms and how cumulative key, value, and key-value sums allow efficient block-level scoring.

4.3. Stage 1: Step-Level Pruning

For the numerator,

$$\tilde{N}_n = \sum_{j=0}^n (1 + q_n^\top k_j) v_j = \sum_{j=0}^n v_j + q_n^\top \sum_{j=0}^n k_j v_j^\top. \quad (4.11)$$

Therefore, the quantities needed for every query position can be precomputed as prefix cumulative sums:

$$V_{\text{cs}}[n] = \sum_{j=0}^n v_j, \quad K_{\text{cs}}[n] = \sum_{j=0}^n k_j, \quad KV_{\text{cs}}[n] = \sum_{j=0}^n k_j v_j^\top. \quad (4.12)$$

Using these cumulative statistics, the Taylor-approximated full attention output is computed as

$$\hat{o}_n^{\text{full}} = \frac{\tilde{N}_n}{\tilde{D}_n} = \frac{V_{\text{cs}}[n] + q_n^\top KV_{\text{cs}}[n]}{(n+1) + q_n^\top K_{\text{cs}}[n]}. \quad (4.13)$$

The important point is that V_{cs} , K_{cs} , and KV_{cs} are built once by a single left-to-right scan over the sequence. After this scan, obtaining $\hat{o}_n^{\text{full}}$ for a query n no longer requires summing over all previous tokens explicitly.

For a candidate screenshot block $\mathcal{B}$ whose token indices form a contiguous interval $[l, r]$, the same cumulative sums give the block statistics by subtraction:

$$\begin{aligned} V_{\mathcal{B}} &= V_{\text{cs}}[r] - V_{\text{cs}}[l-1], \\ K_{\mathcal{B}} &= K_{\text{cs}}[r] - K_{\text{cs}}[l-1], \\ KV_{\mathcal{B}} &= KV_{\text{cs}}[r] - KV_{\text{cs}}[l-1]. \end{aligned} \quad (4.14)$$

where the cumulative terms at index $l-1$ are treated as zero when $l=0$. The Taylor block mass and block numerator are then

$$\tilde{T}_{\mathcal{B},n} = \sum_{j \in \mathcal{B}} (1 + q_n^\top k_j) = |\mathcal{B}| + q_n^\top K_{\mathcal{B}}, \quad (4.15)$$

$$\tilde{N}_{\mathcal{B},n} = \sum_{j \in \mathcal{B}} (1 + q_n^\top k_j) v_j = V_{\mathcal{B}} + q_n^\top K V_{\mathcal{B}}. \quad (4.16)$$

Thus, the Taylor block attention mass and block output are computed as

$$\tilde{A}_{\mathcal{B},n} = \frac{\tilde{T}_{\mathcal{B},n}}{\tilde{D}_n}, \quad \tilde{o}_{\mathcal{B},n} = \frac{\tilde{N}_{\mathcal{B},n}}{\tilde{T}_{\mathcal{B},n}}. \quad (4.17)$$

This turns block scoring into a linear-time procedure: the cumulative statistics are constructed in $O(N)$ time, and each contiguous screenshot block can then be evaluated from two prefix endpoints instead of by repeatedly recomputing attention over the whole prefix. The practical Taylor eviction loss is then

$$\ell_{\text{Taylor}}(\mathcal{B}, n) = \frac{\tilde{A}_{\mathcal{B},n}}{1 - \tilde{A}_{\mathcal{B},n}} \left\| \tilde{o}_{\mathcal{B},n} - \tilde{o}_n^{\text{full}} \right\|_2. \quad (4.18)$$

The final Taylor block score is obtained by normalizing this eviction loss by the Taylor full-output magnitude and averaging across the recent query positions:

$$\mathcal{S}_{\mathcal{B}} = \frac{1}{|\mathcal{Q}|} \sum_{n \in \mathcal{Q}} \frac{\ell_{\text{Taylor}}(\mathcal{B}, n)}{\|\tilde{o}_n^{\text{full}}\|_2 + \epsilon}. \quad (4.19)$$

Thus, computing the Taylor score follows the same scoring logic as exact softmax scoring but replaces repeated full-prefix softmax operations with cumulative statistics. First, build K_{cs} , V_{cs} , and KV_{cs} once with a left-to-right scan. Second, obtain $K_{\mathcal{B}}$, $V_{\mathcal{B}}$, and $KV_{\mathcal{B}}$ for each candidate block by subtracting the two prefix endpoints. Third, compute $\tilde{A}_{\mathcal{B},n}$, $\tilde{o}_{\mathcal{B},n}$, and $\ell_{\text{Taylor}}(\mathcal{B}, n)$ for each recent query $n \in \mathcal{Q}$ without recomputing softmax over the full prefix. Finally, normalize each loss by $\|\tilde{o}_n^{\text{full}}\|_2 + \epsilon$ and average the normalized losses over $\mathcal{Q}$ to obtain $\mathcal{S}_{\mathcal{B}}$. The candidate blocks are ranked in ascending order of $\mathcal{S}_{\mathcal{B}}$, and the lowest-scoring blocks are pruned when their score falls below a safety threshold γ . In this way, γ

4.3. Stage 1: Step-Level Pruning

directly controls the aggressiveness of Stage 1 pruning while preventing removal of blocks whose deletion would cause a large relative perturbation.

Table 4.1 summarizes the correspondence between the exact softmax-based scoring formula and its Taylor approximation. The key difference is the way the score is computed: exact softmax scoring recomputes globally normalized attention terms for each query and block, whereas Taylor scoring approximates the same mass-output-loss pipeline using prefix cumulative statistics and two endpoint subtractions for each contiguous screenshot block.

4.3.1 Matrix-Form View of Softmax and Taylor Scoring

The Stage 1 scoring objective can also be summarized in matrix form. Let $\mathcal{B}$ denote a candidate historical screenshot block, such as a previous GUI screenshot at step $t - 1$, $t - 2$, and so on. Let O^{full} denote the attention-output matrix before removing $\mathcal{B}$, and let $O^{\text{pruned}}(\mathcal{B})$ denote the attention-output matrix after removing $\mathcal{B}$ and recomputing attention over the remaining context. The matrix-form eviction loss is

$$\ell(\mathcal{B}) = \left\| O^{\text{full}} - O^{\text{pruned}}(\mathcal{B}) \right\|_F. \quad (4.20)$$

This loss measures how much the attention output changes when the candidate block is removed. A larger value means that the block has a stronger influence on the model state and should be kept with higher priority.

The corresponding normalized importance score is

$$\mathcal{S}_{\mathcal{B}} = \frac{\ell(\mathcal{B})}{\|O^{\text{full}}\|_F + \epsilon}. \quad (4.21)$$

Here, $\mathcal{S}_{\mathcal{B}}$ is the normalized importance score of block $\mathcal{B}$, and ϵ is a small stability constant. This matrix form is the high-level version of the per-query scoring rule derived above. In practice, O^{full} and $O^{\text{pruned}}(\mathcal{B})$ can be produced by exact softmax attention, while the proposed implementation replaces this expensive exact evaluation with the Taylor-approximated cumulative-statistics score $\mathcal{S}_{\mathcal{B}}$. In both cases, blocks with smaller normalized output perturbation are stronger pruning candidates.

For completeness, the exact softmax attention used in this matrix view can be written with row-wise broadcast division. Define the row-wise division operator $\oslash$ as follows. If $Y \in \mathbb{R}^{N \times d}$ is a matrix and $z \in \mathbb{R}^{N \times 1}$ is a column vector, then

$$(Y \oslash z)_{i,j} = \frac{Y_{i,j}}{z_i}. \quad (4.22)$$

With query, key, and value matrices $Q, K, V \in \mathbb{R}^{N \times d}$, an exact softmax attention-output matrix $O_{\text{softmax}} \in \mathbb{R}^{N \times d}$, and an all-ones vector $\mathbf{1}$, the matrix form of full softmax attention is

$$O_{\text{softmax}} = \left[\exp(QK^{\top})V \right] \oslash \left[\exp(QK^{\top})\mathbf{1} \right], \quad (4.23)$$

where the exponential is applied element-wise. In causal attention, future positions are masked before the exponential is applied, so the row-wise normalization is performed only over the visible prefix for each query.

The Taylor-approximated attention replaces the exponential function in softmax attention with a first-order Taylor expansion to reduce the scoring cost. Applied to the query-key score matrix, this gives

$$\exp(QK^{\top}) \approx \mathbf{1}\mathbf{1}^{\top} + QK^{\top}. \quad (4.24)$$

4.4. Stage 2: Token-Level Pruning and Hybrid Decoding

Substituting this approximation into the matrix attention formula gives

$$O_{\text{Taylor}} = \left[(\mathbf{1}\mathbf{1}^\top + QK^\top)V \right] \oslash \left[(\mathbf{1}\mathbf{1}^\top + QK^\top)\mathbf{1} \right]. \quad (4.25)$$

This can be rearranged into reusable summary terms:

$$O_{\text{Taylor}} = \left[\mathbf{1}(\mathbf{1}^\top V) + Q(K^\top V) \right] \oslash \left[N\mathbf{1} + Q(K^\top \mathbf{1}) \right]. \quad (4.26)$$

Here, $O_{\text{Taylor}} \in \mathbb{R}^{N \times d}$ is the Taylor-approximated attention output, $\mathbf{1}(\mathbf{1}^\top V) \in \mathbb{R}^{N \times d}$ is the summed value information shared across queries, $K^\top V \in \mathbb{R}^{d \times d}$ is a compact key-value summary, $K^\top \mathbf{1} \in \mathbb{R}^{d \times 1}$ is a compact key summary, and $N\mathbf{1} \in \mathbb{R}^{N \times 1}$ is the normalization term from the constant part. This matrix expression is the full-sequence counterpart of the cumulative-statistics formulas used above for causal prefix and block-level scoring.

4.4 Stage 2: Token-Level Pruning and Hybrid Decoding

After Stage 1 step pruning, the reduced sequence is processed with exact softmax prefill. The surviving prefix still contains redundancy, especially inside retained screenshots. Therefore, a second compression stage operates at the token level.

Figure 4.4 shows how the second stage further compresses the trajectory after step pruning. Instead of removing entire screenshots, Stage 2 operates at a finer token level. Tokens with higher estimated importance are retained in the keep set, while low-importance tokens are moved into the prune set and summarized. This allows the method to preserve the most useful local information inside the remaining screenshots while still reducing the number of tokens that require exact attention during decoding.

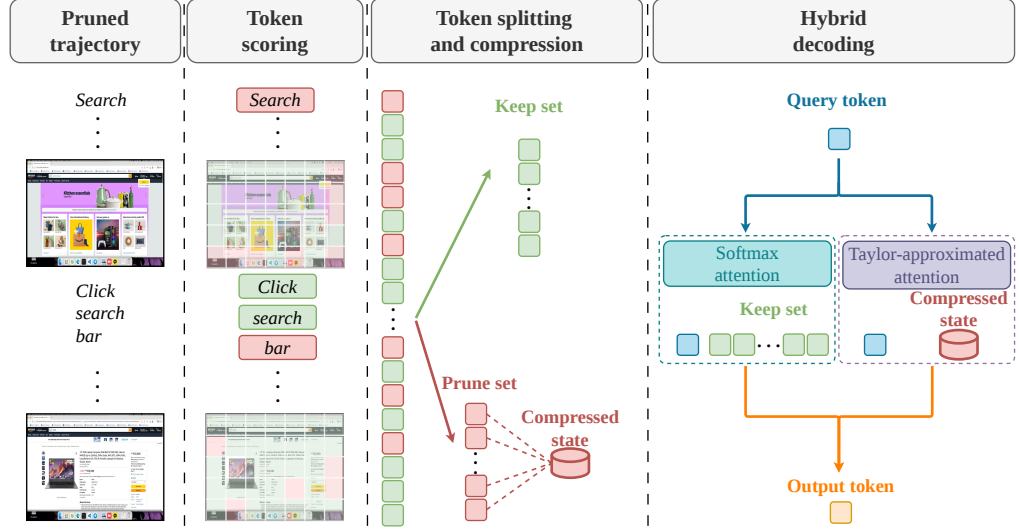


FIGURE 4.4: Stage 2 token-level pruning and hybrid decoding. Important tokens in the reduced context are kept for exact softmax attention, while pruned tokens are summarized into compact statistics.

Let the reduced sequence length after step pruning be N' . Token importance is scored using the recent attention behavior of the model over the reduced prefix. Let K_{total} denote the overall target token budget after both stages. After Stage 1, the sequence has already been shortened from N to N' , so Stage 2 allocates its keep set to match the budget.

Following the general principle of attention-based token pruning [14, 16], the method then forms a keep set $\mathcal{K}$ and a prune set $\mathcal{P}$ using this budget. In general, the set $\mathcal{K}$ contains K_{total} tokens, while $\mathcal{P}$ contains $N' - K_{\text{total}}$ tokens.

This design is appropriate for GUI trajectories because Stage 1 already removes large redundant screenshot blocks, while Stage 2 handles the remaining local redundancy in the reduced context. The compressed summary state for the pruned tokens is created immediately after token scoring so that decoding can proceed without storing all pruned keys and values individually.

4.4. Stage 2: Token-Level Pruning and Hybrid Decoding

As shown in Figure 4.4, decoding uses two information paths. The keep tokens are handled by exact softmax attention, while the pruned tokens contribute through a compact Taylor-approximated correction term. This section derives the corresponding hybrid attention rule.

At decode time, let q denote the new query vector, $\mathcal{K}$ denote the retained token set, and $\mathcal{P}$ denote the prune set after Stage 2 token selection. The query attends to the retained keys and values exactly, while the pruned tokens are represented by compressed summary statistics. The full attention output over the reduced post-step-pruning cache can be written as

$$o(q) = \frac{\sum_{i \in \mathcal{K}} e^{q^\top k_i} v_i + \sum_{j \in \mathcal{P}} e^{q^\top k_j} v_j}{\sum_{i \in \mathcal{K}} e^{q^\top k_i} + \sum_{j \in \mathcal{P}} e^{q^\top k_j}} = \frac{A + B}{C + D}, \quad (4.27)$$

where A and C are the numerator and denominator contributed by keep tokens, and B and D are the corresponding terms contributed by pruned tokens.

Since the pruned tokens are low-importance by construction, their contribution is treated as a small perturbation. A first-order expansion gives

$$\frac{A + B}{C + D} \approx \frac{A}{C} + \frac{B}{C} - \frac{AD}{C^2} = o_{\mathcal{K}} + \frac{B - o_{\mathcal{K}} D}{C}, \quad (4.28)$$

where

$$o_{\mathcal{K}} = \frac{A}{C} = \mathcal{A}(q, K_{\mathcal{K}}, V_{\mathcal{K}}) \quad (4.29)$$

is the exact softmax attention output over the keep set.

The remaining challenge is evaluating B and D without storing the pruned tokens individually. The terms v_{sum} , k_{sum} , $S_{\mathcal{P}}$, and $L_{\mathcal{P}}$ summarize the prune set $\mathcal{P}$ without storing every pruned key and value individually. Applying the Taylor

approximation again to the pruned-token exponentials yields

$$B \approx \sum_{j \in \mathcal{P}} (1 + q^\top k_j) v_j = v_{\text{sum}} + q^\top S_{\mathcal{P}}, \quad (4.30)$$

$$D \approx \sum_{j \in \mathcal{P}} (1 + q^\top k_j) = L_{\mathcal{P}} + q^\top k_{\text{sum}}, \quad (4.31)$$

where

$$v_{\text{sum}} = \sum_{j \in \mathcal{P}} v_j, \quad k_{\text{sum}} = \sum_{j \in \mathcal{P}} k_j, \quad S_{\mathcal{P}} = \sum_{j \in \mathcal{P}} k_j v_j^\top, \quad L_{\mathcal{P}} = |\mathcal{P}|. \quad (4.32)$$

The final hybrid decoding rule becomes

$$o \approx o_{\mathcal{K}} + \frac{(v_{\text{sum}} + q^\top S_{\mathcal{P}}) - o_{\mathcal{K}} (L_{\mathcal{P}} + q^\top k_{\text{sum}})}{C}. \quad (4.33)$$

This expression keeps the exact softmax computation on the most important tokens and adds a cheap correction term for the pruned ones. Only four fixed-size tensors are stored for the pruned state, so the contribution of all pruned tokens is summarized compactly.

To improve numerical stability, the method uses a simple gating rule for the Taylor correction. In the exact softmax attention, the denominator contribution of any token set is always positive because it is a sum of exponential terms. For the prune set, this exact denominator contribution would be $\sum_{j \in \mathcal{P}} \exp(q^\top k_j)$. After the first-order Taylor approximation, this term is replaced by the linear expression

$$D = L_{\mathcal{P}} + q^\top k_{\text{sum}} \quad (4.34)$$

where $L_{\mathcal{P}} = |\mathcal{P}|$ is the number of pruned tokens and k_{sum} is the sum of their key

4.5. Complexity Analysis

vectors. This quantity represents the approximate denominator mass contributed by the pruned tokens.

Although the exact denominator mass is strictly positive, the Taylor approximation can become negative for some query vectors because the inner product term $q^\top k_{\text{sum}}$ may be sufficiently negative. A negative denominator correction is not meaningful as attention mass and can make the correction term unstable. Therefore, the decoder applies the correction only when the approximate pruned mass is valid:

$$o = \begin{cases} o_{\mathcal{K}} + \frac{(v_{\text{sum}} + q^\top S_{\mathcal{P}}) - o_{\mathcal{K}} (L_{\mathcal{P}} + q^\top k_{\text{sum}})}{C}, & \text{if } L_{\mathcal{P}} + q^\top k_{\text{sum}} \geq 0, \\ o_{\mathcal{K}}, & \text{otherwise.} \end{cases} \quad (4.35)$$

In other words, when the Taylor approximation produces a plausible non-negative prune-set mass, the model uses the hybrid output with the correction term. When the approximation is unreliable, the correction is suppressed and the model falls back to the exact keep-only output $o_{\mathcal{K}}$. This fallback is conservative: it may ignore the summarized pruned tokens for that decode step, but it avoids injecting an unstable correction into the output.

4.5 Complexity Analysis

Table 4.2 compares the three inference strategies by separating the cost into prefilling, scoring, and decoding. Here, N is the original context length, N' is the reduced length after Stage 1 step pruning, and K_{total} is the number of tokens retained for exact decode-time attention after compression.

Component	Full cache	Token-only pruning	Proposed method
Prefilling	$O(N^2)$	$O(N^2)$	$O(N^2)$
Scoring	–	$O(N)$	$O(N)$
Decoding	$O(N)$	$O(K_{\text{total}})$	$O(K_{\text{total}})$

TABLE 4.2: Theoretical complexity comparison of full-cache inference, token pruning, and the proposed multi-stage context pruning pipeline.

Full Cache. In full-cache inference, no pruning is applied. During prefilling, the model processes the entire sequence of length N with exact causal self-attention. Since each token can attend to previous tokens, the number of query-key interactions grows quadratically with the sequence length, so the prefill cost is $O(N^2)$. There is no scoring stage because no token or step is selected for removal, which is why the scoring entry is marked as “–”. During decoding, every new query attends to all cached tokens from the original context, so the per-token decoding cost scales linearly with the full cache size, giving $O(N)$.

Token-only Pruning. Token-only pruning methods compress the KV cache only after the full context has already been processed. Therefore, they still pay the same exact prefill cost as full-cache inference, namely $O(N^2)$. After prefilling, token importance is computed over the sequence to choose which KV entries should remain in the cache. For fixed hidden dimension, this token-scoring pass is linear in the number of candidate tokens, so the scoring cost is $O(N)$. Once the cache is reduced, decoding attends only to the retained K_{total} tokens, so the per-token decoding cost becomes $O(K_{\text{total}})$ instead of $O(N)$.

Proposed Method. The proposed method differs from token-only pruning because it removes low-importance historical screenshot blocks before exact

4.5. Complexity Analysis

prefilling. Stage 1 scoring uses the Taylor surrogate and cumulative statistics, so candidate screenshot blocks can be scored in a linear pass over the original sequence, giving the additional $O(N)$ term. After this step reduction, exact prefill is applied only to the shorter retained sequence of length N' , so the dominant prefill term becomes $O(N'^2)$ rather than $O(N^2)$. This gives the table entry $O(N'^2)$. The scoring row is also $O(N)$ for the proposed method because the step-level Taylor scoring is linear in the original sequence length. During decoding, exact softmax attention is applied only to the keep set $\mathcal{K}$ with size K_{total} , while the prune set $\mathcal{P}$ is represented by fixed-size summary terms such as v_{sum} , k_{sum} , $S_{\mathcal{P}}$, and $L_{\mathcal{P}}$. Therefore, the leading decode-time cost is $O(K_{\text{total}})$.

The key difference is where compression occurs. Full-cache inference performs no compression and remains expensive in both prefill and decoding. Token pruning reduces only decode-time cost as it acts after the full prefill. The proposed method reduces both prefill and decode-time cost by pruning screenshot blocks before prefill and then compressing remaining tokens before decoding.

Chapter 5

Experimental Results and Discussion

5.1 Benchmarks

The main offline benchmark used in this thesis is AGENTNETBENCH, following the evaluation setting introduced in the OpenCUA paper [6]. AgentNetBench is an offline computer-use agent (CUA) benchmark. It is designed to make evaluation stable and fast without requiring a live environment, and it evaluates predicted low-level actions against gold trajectories. The benchmark covers Windows and macOS tasks and is particularly suitable for this thesis because it exposes long multi-step GUI trajectories in which historical screenshots create substantial KV cache overhead.

To position the study within the broader GUI agent literature, this chapter also includes OSWORLD as a reference benchmark [31]. OSWorld is a real-computer benchmark for multimodal agents that supports Linux environments and includes open-ended tasks over real web and desktop applications. The original OSWorld paper emphasizes realistic task setups and execution-based evaluation, making it a valuable complement to AgentNetBench when discussing the generality of GUI agent methods beyond a single benchmark protocol.

5.2 Metrics

This thesis reports two evaluation metrics: Step Success Rate [6, 32] and Task Success Rate [31, 33]. Step Success Rate measures whether the predicted low-level action at each interaction step matches the reference action. It is the main metric for offline trajectory evaluation because it directly reflects whether the model chooses the correct next GUI operation given the current context.

Task Success Rate measures whether the complete task is successfully finished. This metric evaluates the end-to-end outcome of an agent trajectory, so it is stricter than step-level matching and better reflects the practical usability of a GUI agent in an interactive environment. Together, these two metrics capture both local action accuracy and final task completion performance.

5.3 Models

The two agent backbones considered in this chapter are Qwen3VL 8B and EvoCUA 8B. Qwen3VL 8B is the dense 8B model from the Qwen3-VL family [1]. It is a strong vision-language model pretrained for both natural language tasks and GUI agent workflows, making it a natural backbone choice for testing long-context GUI agent inference under KV-cache compression.

EvoCUA 8B is the 8B variant of EvoCUA [2], a CUA model trained with an evolving learning framework built on scalable synthetic experiences. Using both backbones is useful because it allows the proposed pruning method to be evaluated on both a strong general multimodal model and a more task-specialized CUA.

5.4 Baselines

The main baseline methods are SnapKV and GUI-KV. SnapKV [16] is a training-free KV-cache compression method that uses an observation window near the end of the prompt to identify important cached positions for each attention head. It is a strong token-level baseline because it reduces decode-time KV cost without requiring retraining.

GUI-KV [30] is a more task-relevant baseline designed specifically for GUI agents. It remains training-free, but it adds GUI-specific inductive bias through spatial-saliency guidance and temporal redundancy scoring so that redundant historical screenshots can be pruned more effectively. This makes GUI-KV a particularly meaningful comparison point for the proposed method, since both approaches aim to improve efficiency on screenshot-heavy, long-horizon GUI tasks while preserving action quality.

5.5 Quantitative Results

All experiments in this chapter are conducted using NVIDIA RTX A6000 GPUs. Unless otherwise specified, the reported accuracy results use the same hardware environment across methods so that the comparisons focus on the effect of the KV-cache pruning strategy rather than hardware differences. The proposed method uses image-only pruning in Stage 1. It removes historical screenshot blocks before exact prefilling while preserving the associated textual reasoning and action history. Stage 2 then performs token-level compression on the retained reduced context, where any low-importance token, including redundant text tokens, can be compressed into the summary state.

For Qwen3-VL 8B and EvoCUA 8B, the compressed summary state has a fixed size of 65 tokens and is added separately as an overhead outside the pruning budget. It is negligible compared with the full 16K–32K input length in GUI setting. At a 20% KV budget, the compressed state accounts for only about 1.02%–2.03% of the retained tokens and excluding it has minimal impact on the performance. When the summary state is included in the budget calculation, the simulated performance is estimated to decrease by approximately 3%–4% at the 20% KV budget, 0.2%–0.5% at the 40% budget, and 0.1%–0.2% at the 60% budget.

Table 5.1 on the next page reports the main results on AGENTNETBENCH under different KV-cache budgets. For Qwen3VL 8B, the proposed method is most effective under aggressive compression. At the 20% KV budget, it reaches 78.3% overall success rate, which is much higher than SnapKV at 57.7% and GUI-KV at 54.1%. This shows that step-aware screenshot pruning is especially useful when the cache budget is very small. At the 40% budget, the proposed method still performs best with 81.2%, compared with 78.2% for SnapKV and

5.5. Quantitative Results

TABLE 5.1: Success rates on AGENTNETBENCH under different KV-cache budgets. The action categories are coordinate, content, and function, and the KV budget denotes the percentage of KV cache retained during inference. For each model, budget, and metric, **boldface indicates the best score** among SnapKV, GUI-KV, and Proposed, excluding Full Cache.

Model	KV Budget	Method	Success Rate (%) $\uparrow$			
			Coord.	Content	Func.	Overall
Qwen3VL 8B	Full	Full Cache	81.9	79.9	87.6	82.0
		SnapKV	58.2	47.3	72.2	57.7
		GUI-KV	53.4	39.8	86.6	54.1
	20%	Proposed	78.8	72.7	84.5	78.3
		SnapKV	78.4	72.1	87.6	78.2
		GUI-KV	77.3	74.8	87.6	77.8
		Proposed	81.0	79.9	84.5	81.2
	40%	SnapKV	80.8	75.3	87.6	80.6
		GUI-KV	80.0	79.3	87.6	80.1
		Proposed	82.2	80.4	83.5	82.0
	60%	SnapKV	66.5	59.2	53.6	64.4
		SnapKV	55.6	55.4	50.5	55.1
		GUI-KV	48.9	52.3	49.5	49.4
EvoCUA 8B	20%	Proposed	58.8	54.2	47.4	57.2
		SnapKV	64.7	58.7	53.6	63.0
		GUI-KV	65.5	58.5	52.6	63.4
	40%	Proposed	65.5	58.1	55.7	63.6
		SnapKV	66.7	59.7	52.6	64.5
		GUI-KV	63.1	58.6	49.5	61.3
	60%	Proposed	67.4	60.5	47.4	64.8

77.8% for GUI-KV. At the 60% budget, it reaches 82.0%, matching the full-cache result while retaining only part of the KV cache.

For EvoCUA 8B, the same trend is observed, although the performance gaps are smaller. At the 20% budget, the proposed method obtains 57.2%, outperforming SnapKV at 55.1% and GUI-KV at 49.4%. At the 40% budget, it reaches 63.6%, slightly higher than SnapKV and GUI-KV. At the 60% budget, the proposed method achieves 64.8%, which is slightly higher than the full-cache result of 64.4% and SnapKV at 64.5%. This slight gain at a moderate budget can be explained by the nature of GUI trajectories. Long observation histories often include redundant information that are no longer relevant to the current action. Keeping all of context can introduce noise in the model input and weaken attention to the most informative evidence for the next decision. Pruning at 60% typically removes low important or duplicated history while preserving task-critical screenshots and tokens, which can reduce distraction and improve action prediction. Similar behavior has been reported in previous works [29, 30], where moderate compression may slightly increase performance, whereas overly aggressive pruning can remove necessary context and hurt results.

Overall, the table shows that token-only cache compression methods such as SnapKV and GUI-KV become less reliable when the KV budget is small, especially for Qwen3VL 8B. In contrast, the proposed method first removes less informative screenshot blocks before token-level compression, which better matches the structure of GUI trajectories. This explains why the proposed method maintains stronger overall performance across both backbones and all tested KV-cache budgets.

5.5. Quantitative Results

TABLE 5.2: Success rates on OSWORLD under different KV-cache budgets. Tasks are grouped into OS, Office, Daily, Professional, and Workflow categories. The KV budget denotes the percentage of KV cache retained during inference. For each model, budget, and task category, **boldface indicates the best score** among SnapKV, GUI-KV, and Proposed, excluding Full Cache.

Model	KV Budget	Method	Success Rate (%) $\uparrow$					
			OS	Office	Daily	Profess.	Workflow	Overall
Qwen3VL 8B	Full	Full Cache	41.7	33.2	48.5	42.9	16.1	34.0
		SnapKV	12.5	5.2	7.5	6.1	3.0	5.7
		GUI-KV	16.7	3.4	11.5	2.0	3.0	5.8
	20%	Proposed	25.0	20.5	44.8	51.0	13.3	28.3
		SnapKV	16.7	19.6	34.3	30.6	3.1	19.8
		GUI-KV	16.7	12.9	20.5	14.3	3.3	12.5
		Proposed	29.2	28.1	46.9	46.9	9.6	30.1
	40%	SnapKV	25.0	28.1	47.2	38.8	5.5	27.6
		GUI-KV	16.7	11.1	20.5	18.4	7.3	13.5
		Proposed	37.5	31.5	46.1	49.0	11.4	32.3
	60%	SnapKV	69.6	39.2	57.0	73.5	23.3	45.6
		GUI-KV	30.4	22.1	17.9	22.9	11.7	19.2
		Proposed	54.2	28.1	45.5	68.1	15.1	35.5
EvoCUA 8B	20%	SnapKV	41.7	32.5	46.1	51.0	21.8	35.8
		GUI-KV	50.0	34.8	41.0	55.3	24.6	37.2
		Proposed	58.3	33.2	55.7	73.5	20.2	41.9
	40%	SnapKV	58.3	39.2	58.8	69.4	25.7	45.4
		GUI-KV	62.5	40.9	57.1	69.4	24.3	45.4
		Proposed	62.5	38.4	56.4	77.6	25.0	45.8
	60%	SnapKV	69.6	39.2	57.0	73.5	23.3	45.6
		GUI-KV	30.4	22.1	17.9	22.9	11.7	19.2
		Proposed	54.2	28.1	45.5	68.1	15.1	35.5

Table 5.2 reports the corresponding results on OSWORLD, which is a more open-ended and execution-oriented benchmark than AGENTNETBENCH. For Qwen3VL 8B, the proposed method is again much more robust under aggressive

compression. At the 20% KV budget, it achieves 28.3% overall success rate, compared with only 5.7% for SnapKV and 5.8% for GUI-KV. This large gap shows that directly applying token-only cache compression can remove too much useful GUI history when the available cache is very small. In contrast, the proposed method first removes redundant historical screenshot blocks and then performs token-level compression, allowing the model to retain a more useful reduced context. At the 40% and 60% budgets, the proposed method reaches 30.1% and 32.3% overall success rate, respectively, outperforming both baselines and approaching the full-cache result of 34.0%.

For EvoCUA 8B, the proposed method also performs competitively across budgets. At the 20% budget, it obtains 35.5% overall success rate, substantially higher than SnapKV at 19.2% and GUI-KV at 21.0%. This indicates that the multi-stage pruning strategy is especially helpful when compression is severe. At the 40% budget, the proposed method reaches 41.9%, again outperforming both baselines. At the 60% budget, it achieves the best overall result of 45.8%, slightly above the full-cache result of 45.6% and above both SnapKV and GUI-KV at 45.4%. The category-level results show a similar pattern: the proposed method is particularly strong on OS and Professional tasks, suggesting that preserving the most relevant visual history helps with tasks that require tracking desktop state and multi-step application context.

Overall, the OSWORLD results confirm the main trend observed on AGENT-NETBENCH. The proposed method is most valuable when the KV-cache budget is limited, because it uses the structure of GUI trajectories rather than treating the prompt as a flat token sequence. By pruning less informative screenshots before token-level cache compression, the method preserves more of the context needed for action prediction under tight memory budgets. The remaining gap to

5.5. Quantitative Results

full-cache performance on Qwen3VL 8B also shows that OSWorld remains challenging under compression, especially for workflow-style tasks where long-range context and precise state tracking are important.

5.6 Qualitative Results

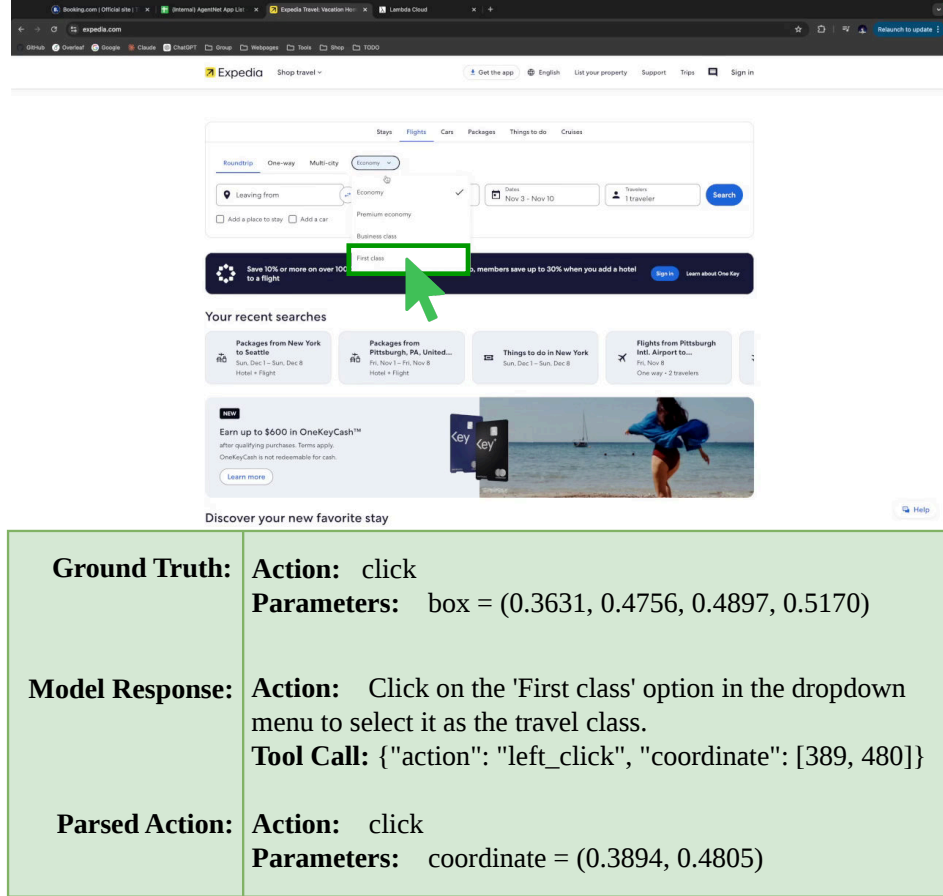


FIGURE 5.1: Successful click prediction from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model selects the First class option with the correct action type and target location.

Figures 5.1 and 5.2 on the next page show representative successful predictions extracted by running the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH. This setting corresponds to the most aggressive compression case in the main AGENTNETBENCH comparison, so the examples illustrate what information can still be preserved when only a small

5.6. Qualitative Results

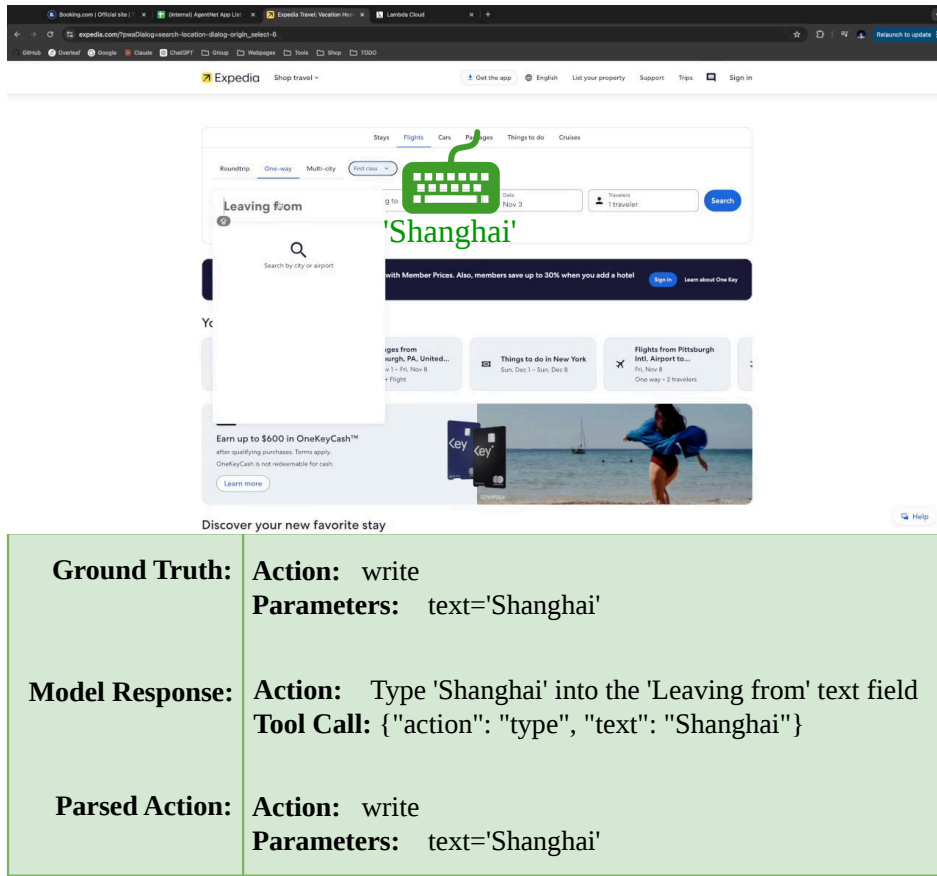


FIGURE 5.2: Successful text-entry prediction from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model writes Shanghai into the intended Leaving from field.

fraction of the KV cache is retained. The first example demonstrates that the model can still recover a precise click action after compression. The ground-truth action is to click the First class option in the travel-class dropdown, and the parsed model coordinate falls inside the target region. This suggests that the retained visual context is sufficient for localizing important GUI elements when irrelevant screenshot history is removed.

The second example shows a successful text-entry action. The model correctly predicts both the action type and the text content, writing “Shanghai” into the intended Leaving from field. This indicates that the pruning process can preserve the task instruction and the relevant state information needed for content-based actions, not only coordinate-based clicks.

Together, these examples show that the proposed method can preserve different types of GUI-agent behavior after context pruning. It can maintain spatial grounding for click actions and retain enough textual and task information for write actions, which supports the effectiveness of pruning redundant screenshot history while keeping the context needed for the next GUI operation.

5.7 Wall-Time and FLOPs Analysis

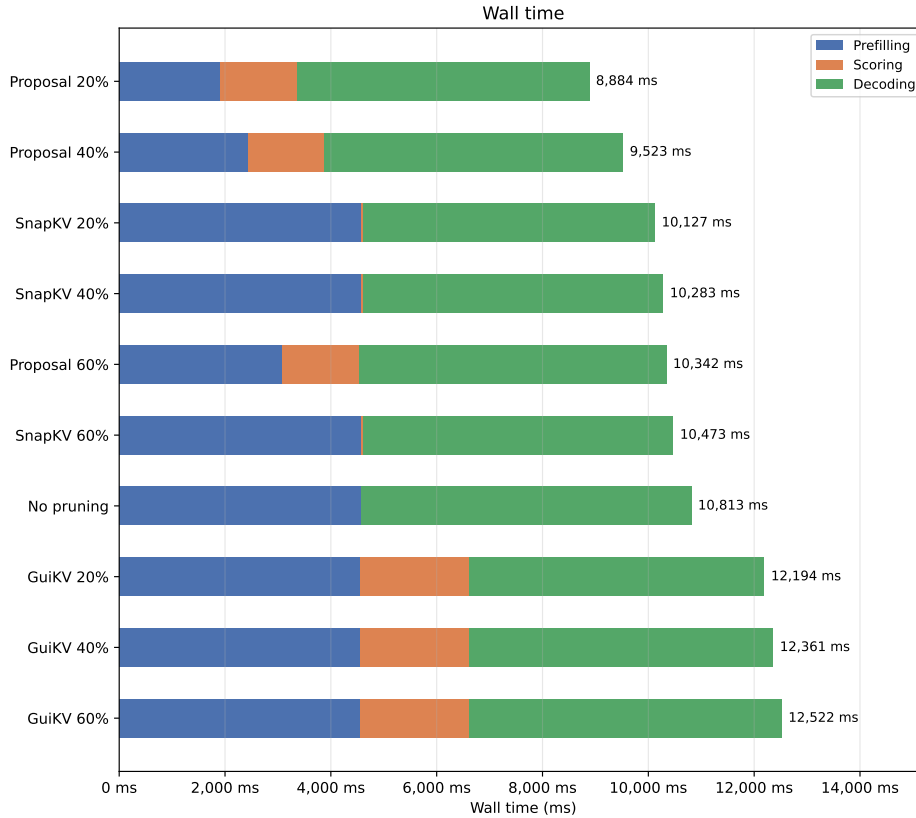


FIGURE 5.3: Wall-time comparison across pruning methods and KV-cache budgets. Runtime is decomposed into prefilling, scoring, and decoding stages.

Figures 5.3 and 5.4 on the next page compare the practical wall time and theoretical FLOPs of different pruning methods. The wall-time measurements are collected on a single NVIDIA RTX A6000 GPU and are decomposed into prefilling, scoring, and decoding costs. The proposed method gives the lowest wall time at the 20% and 40% budgets, requiring 8,884 ms and 9,523 ms, respectively. These results are faster than SnapKV at the same budgets, which requires 10,127 ms and 10,283 ms. At the 60% budget, the proposed method takes 10,342 ms, which

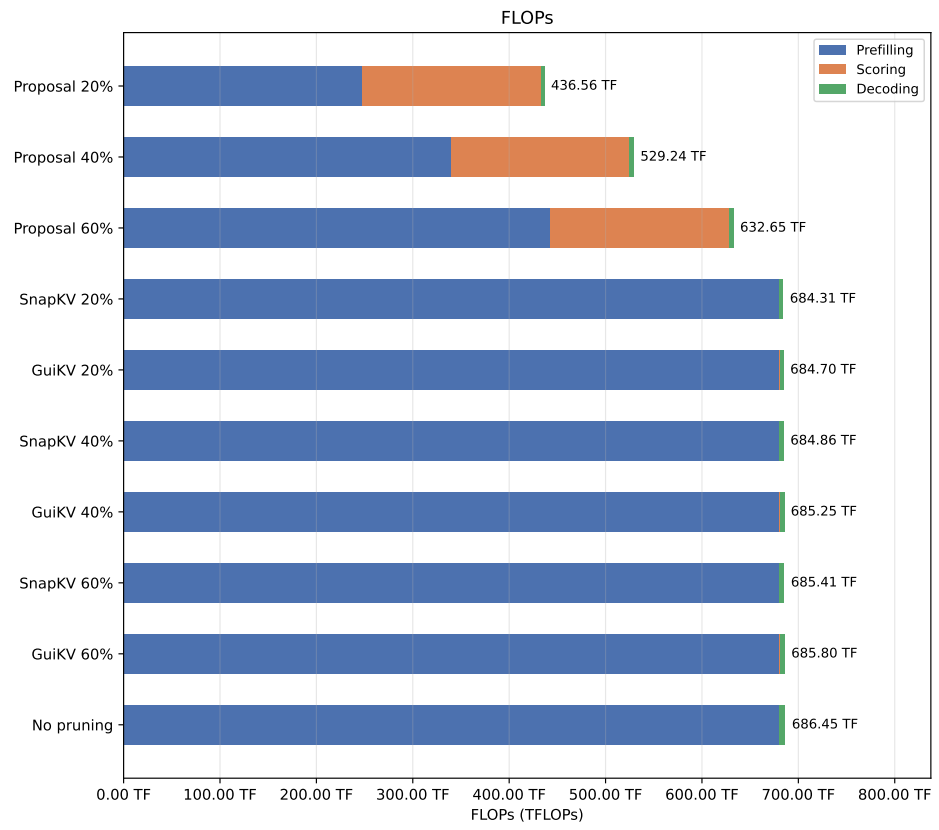


FIGURE 5.4: Theoretical FLOPs comparison across pruning methods and KV-cache budgets. The stacked bars separate prefilling, scoring, and decoding costs.

5.7. Wall-Time and FLOPs Analysis

remains slightly faster than SnapKV at 10,473 ms and the no-pruning baseline at 10,813 ms. GUI-KV is the slowest method across all budgets, reaching 12,194 ms, 12,361 ms, and 12,522 ms at the 20%, 40%, and 60% budgets. Although the FLOPs added by GUI-KV scoring are small, its scoring stage is relatively long in wall-clock time because the pipeline is more affected by CPU-side dispatch, control flow, and data-management overhead. This shows that runtime is not determined only by arithmetic FLOPs.

The FLOPs comparison shows a clearer advantage for the proposed method. The proposed method uses 436.56 TFLOPs at the 20% budget, 529.24 TFLOPs at the 40% budget, and 632.65 TFLOPs at the 60% budget. These values are all below the no-pruning baseline of 686.45 TFLOPs. In contrast, SnapKV and GUI-KV remain close to the no-pruning cost, ranging from 684.31 to 685.80 TFLOPs across budgets, because they mainly compress the cache after the full context has already been prefetched. The proposed method reduces FLOPs because it removes redundant historical screenshot blocks before exact prefill, so the most expensive quadratic attention computation is applied to a shorter sequence. The reduction is largest at the 20% budget, where the retained context is smallest.

The apparent mismatch between FLOPs and wall time comes from the different execution characteristics of prefilling and decoding. In the evaluated setting, prefilling processes a long prompt of about 23,000 tokens, whereas decoding generates only about 200 tokens. Therefore, prefilling dominates the theoretical FLOPs because causal self-attention over the prompt scales quadratically with the prefill length. However, prefilling is also highly parallelizable on the GPU: the long prompt can be processed with large batched matrix operations that use GPU compute units efficiently. Decoding has much fewer FLOPs, but it is autoregressive and sequential. Each generated token requires a separate step

that repeatedly reads the retained KV cache and launches smaller operations. As a result, decoding can take a large fraction of wall-clock time even when its theoretical FLOPs are much smaller than the prefill FLOPs.

Overall, the wall-time and FLOPs results highlight the trade-off introduced by the multi-stage design. Reducing prefill length lowers the dominant theoretical FLOPs, while reducing the retained KV cache also helps the memory-bound decoding stage. This explains why the proposed method is most effective under tighter budgets. At low budgets, more redundant visual history is removed before prefill and fewer tokens remain for decoding, so the savings outweigh the additional Taylor scoring cost. As the budget increases, the retained sequence becomes larger, the prefill and decoding costs rise, and the wall-time gap between the proposed method and token-only pruning becomes smaller. Therefore, the proposed pruning strategy is particularly useful when the goal is to operate under limited KV-cache budgets while preserving the accuracy trends shown in the quantitative results.

5.8 Hyperparameter Settings

TABLE 5.3: Hyperparameter sensitivity of Qwen3VL 8B on AGENTNETBENCH. We report the overall success rate under different KV-cache budgets, observation window sizes W , and normalized eviction thresholds γ . **Boldface indicates the best score** for each KV-cache budget.

Budget	W	$\gamma = 0.05$	$\gamma = 0.06$	$\gamma = 0.07$	$\gamma = 0.08$
20%	4	77.8	77.6	77.5	76.6
	8	78.1	78.3	78.3	76.5
	16	74.1	73.6	74.7	73.9
40%	4	81.5	81.4	81.5	81.3
	8	81.1	81.2	81.2	80.8
	16	81.4	81.4	81.5	81.5
60%	4	81.7	81.8	81.9	81.7
	8	81.6	82.0	82.0	81.8
	16	81.6	81.7	81.7	81.6

Table 5.3 shows that the proposed method is not overly sensitive to small changes in the normalized eviction threshold when the observation window is chosen appropriately. Across the three KV-cache budgets, $W = 8$ gives consistently strong results and achieves the best or tied-best performance in several settings. In particular, the setting $W = 8$ and $\gamma = 0.06$ reaches 78.3% at the 20% KV budget, 81.2% at the 40% budget, and 82.0% at the 60% budget.

Based on this trade-off, this thesis uses $W = 8$ and $\gamma = 0.06$ as the default hyperparameter setting. The window size $W = 8$ provides enough recent query positions to estimate screenshot-block importance reliably, while avoiding the extra cost and weaker low-budget performance observed with a larger window

such as $W = 16$. The threshold $\gamma = 0.06$ is also a stable middle choice: it avoids overly conservative pruning at lower thresholds and avoids overly aggressive pruning at higher thresholds.

5.9 Ablation Study

TABLE 5.4: Ablation study on AGENTNETBENCH with Qwen3VL 8B. The table reports overall step success rate under different KV-cache budgets. The symbols ✓ and × indicate whether Stage 1 step pruning and Stage 2 hybrid decoding are enabled. **Boldface indicates the best score** for each KV-cache budget.

Variant	Step	Hybrid	20%	40%	60%
Baseline	×	×	57.7	78.2	80.6
Hybrid decoding only	×	✓	61.3	80.6	81.9
Step pruning only	✓	×	73.2	80.7	81.6
Proposed	✓	✓	78.3	81.2	82.0

Table 5.4 separates the contribution of the two main components in the proposed framework. The baseline corresponds to token-only compression without Stage 1 step pruning or hybrid decoding. Adding hybrid decoding alone improves the result at all budgets, showing that summarized pruned-token information is useful during decoding. This suggests that even after tokens are removed from the KV cache, their compressed semantic information can still help the model preserve action-relevant context. Adding step pruning alone gives a much larger gain at the 20% budget, increasing performance from 57.7% to 73.2%, which indicates that removing redundant historical screenshots before prefilling is especially important under severe compression. The improvement from step pruning is smaller at higher budgets, because more context can already be retained by the token-level compressor. Combining both components gives the strongest results at the 20% and 40% budgets and remains competitive at the 60% budget, confirming that the two stages are complementary.

5.10 Performance of Different Step Pruning Approaches

TABLE 5.5: Overall success rate of Qwen3VL 8B on AGENTNET-BENCH under different Stage 1 pruning approaches. Random pruning removes historical screenshot blocks without an importance signal, far-to-near pruning removes older screenshots first, and Proposed uses the Taylor-approximated perturbation score for image-only pruning. **Boldface indicates the best score** for each KV-cache budget.

Stage 1 Pruning	20% KV	40% KV	60% KV
Random Pruning	76.9	80.1	81.1
Far-to-Near Pruning	76.9	80.1	82.0
Proposed	78.3	81.2	82.0

Table 5.5 compares simple pruning approaches with the proposed importance-based pruning strategy on AGENTNETBENCH. Random pruning provides a non-informative baseline, while far-to-near pruning tests the simple assumption that older screenshots are less important. The proposed method performs best at the 20% and 40% budgets, reaching 78.3% and 81.2%, respectively, and ties the best result at the 60% budget with 82.0%. These results show that estimating screenshot-block importance is more reliable than removing blocks randomly or relying only on temporal distance from the current step. The advantage is most visible under the 20% budget, where the cache is highly constrained and each pruning decision has a larger effect on the remaining context. In this regime, random and far-to-near pruning both reach 76.9%, while the proposed method improves the result by 1.4 percentage points. This indicates that useful GUI evidence is not determined solely by recency: an older screenshot may still contain information about a menu state, selected option, or previously entered content that is relevant to the next action. At the 60% budget, all approaches

5.10. Performance of Different Step Pruning Approaches

become closer because enough visual history can be retained even when the pruning order is simple. Therefore, the main benefit of the proposed approach is its robustness under aggressive compression, where it can remove redundant screenshot blocks while preserving the historical visual evidence most likely to affect the model output.

5.11 Performance of Scoring Approximation and Pruning Scope

TABLE 5.6: Performance of Stage 1 scoring approximation and pruning scope variants using Qwen3VL 8B on AGENTNETBENCH. The Approximation column identifies whether the Stage 1 score uses Taylor approximation, the Image and Text columns indicate whether screenshot tokens and associated text tokens are eligible for pruning. **Boldface indicates the best score** for each KV-cache budget.

Variant	Approximation	Image	Text	20% KV	40% KV	60% KV
Proposed (Softmax Scoring)	×	✓	×	78.6	81.5	82.0
Proposed (Image + Text)	✓	✓	✓	68.1	71.0	81.0
Proposed (Image only)	✓	✓	×	78.3	81.2	82.0

Table 5.6 separates the effect of the scoring approximation from the effect of pruning scope. The proposed image-only pruning is the version we use for quantitative results. The proposed softmax scoring uses exact attention-based step scoring and gives the best result at the 20% and 40% budgets, reaching 78.6% and 81.5%, respectively. The proposed image-only pruning with approximation is close to this upper reference, reaching 78.3%, 81.2%, and 82.0% at the three budgets, while avoiding high cost of softmax-based scoring. The image-only variants remove historical screenshot blocks but keep the associated text tokens. In contrast, the image-plus-text variant removes the entire selected historical step. Its much lower performance at the 20% and 40% budgets, 68.1% and 71.0%, shows that textual history remains important even when visual history is redundant. Overall, these results support the design choice of pruning visual history separately from textual history and using the Taylor-approximated perturbation score as a practical criterion for selecting redundant screenshot blocks.

5.12 Performance with Varying Numbers of Screenshots

TABLE 5.7: Performance on AGENTNETBENCH with varying numbers of past steps. **Boldface indicates the best score** for each model.

Model	2	3	4	5	7	10	15
Qwen3VL 8B	78.0	79.2	81.9	82.0	81.9	81.5	81.7
EvoCUA 8B	63.0	63.3	63.1	64.4	63.7	63.7	63.7

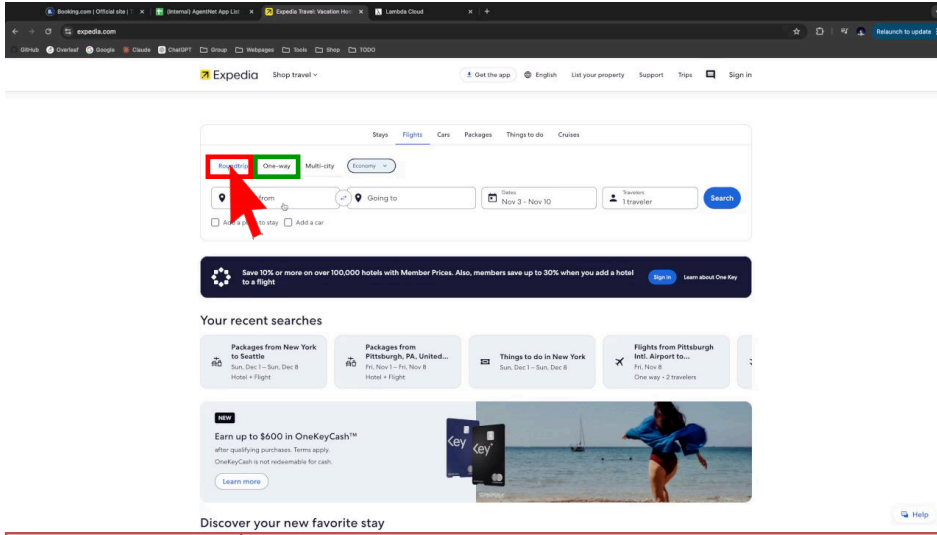
Table 5.7 shows that both models benefit from adding a moderate amount of visual history, but the gain saturates after a certain point. The strongest result occurs at five past steps for both backbones, reaching 82.0% for Qwen3VL 8B and 64.4% for EvoCUA 8B. This indicates that a short local history is useful for GUI agents because the previous screens often contain information about recently opened menus, typed text, selected options, or intermediate task states that are not fully visible from the current screenshot alone.

When the history is extended beyond five screenshots, performance stays nearly flat or slightly decreases, suggesting that very long histories add redundant visual context rather than consistently useful information. In GUI trajectories, consecutive screenshots are often highly similar because many actions change only a small region of the interface. As a result, adding more past screenshots can increase the token length without adding much new evidence for the next action. Older screenshots may also describe interface states that are no longer active, which can distract the model from the current GUI state or introduce conflicting visual cues.

This observation supports the design of the proposed pruning method. Instead of assuming that all historical screenshots should be preserved, the method treats

the visual history as a structured sequence with unequal importance across steps. A moderate history length provides enough temporal context for action prediction, while step-level pruning can remove redundant or stale screenshots when the context becomes longer. Therefore, the results in this table motivate both parts of the proposed framework: keeping useful recent trajectory information and compressing the visual history before it becomes an unnecessary KV-cache burden.

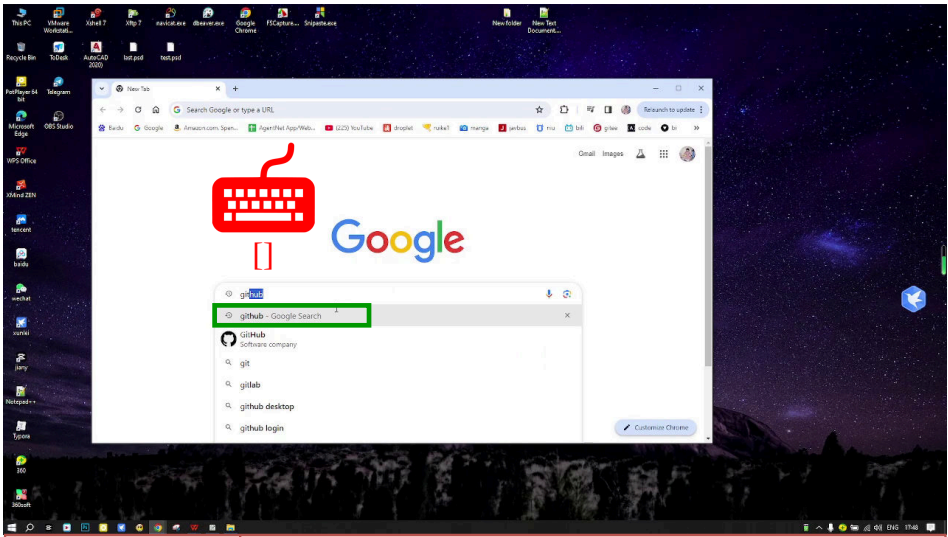
5.13 Discussion and Failure Cases



Ground Truth:	Action: click Parameters: box = (0.3645, 0.2944, 0.4138, 0.3224)
Model Response:	Action: Click on the 'One-way' option to change the flight search to a one-way trip. Tool Call: {"action": "left_click", "coordinate": [285, 320]}
Parsed Action:	Action: click Parameters: coordinate = (0.2853, 0.3203)
Why it failed:	Grounding error: Correct target identified but click locations miss the 'One-way' button. This suggests less tokens can make the grounding become less stable.

FIGURE 5.5: Grounding error from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the target is identified correctly, but the click location misses the one-way button.

Figures 5.5 and 5.6 on the next page show two representative failure cases extracted from the same setting as the successful examples: the proposed method



Ground Truth:	Action: click Parameters: box = (0.2252, 0.6096, 0.3885, 0.0432)
Model Response:	Action: Press 'Enter' to execute the search for 'github'. Tool Call: {"action": "key", "key": "enter"}
Parsed Action:	Action: press Parameters: []
Why it failed:	Template error: The parser expects {"action": "hotkey", "keys": ["enter"]} but the model generated the wrong template, singular "key" instead of list template "keys": [], the "key" is unrecognized, so the action parameters become the empty list. Understanding error: The 'github' word is not completed, the Ground Truth action to click on the box to autocomplete first before the 'enter' to search. The model makes wrong prediction due to reduced context.

FIGURE 5.6: Template and understanding error from the proposed method with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH: the model generates an invalid key template and submits the query too early.

5.13. Discussion and Failure Cases

with a 20% KV-cache budget using Qwen3VL 8B on AGENTNETBENCH. Because this setting applies aggressive cache compression, these cases highlight the types of errors that can remain even when the overall success rate is strong. In the first case, the model correctly recognizes that the next action should select the one-way flight option, but the parsed click coordinate falls outside the target button. This is mainly a grounding error: the semantic intent is preserved, but the spatial precision needed for coordinate prediction becomes less stable after reducing the visual context. This failure suggests that aggressive pruning can weaken fine-grained localization even when the correct UI element is still described in the model response.

The second case contains both a tool-template error and an understanding error. The ground-truth action is to click the autocomplete suggestion box for “github”, but the model predicts pressing Enter. In addition, the generated tool call uses the template “key” instead of the expected hotkey format, so the parser cannot recover a valid key parameter and returns an empty parameter list. This example shows that pruning can affect not only visual grounding, but also action-format consistency and short-context reasoning about the current UI state.

Overall, these cases show that context pruning can fail in two important ways. First, it can preserve the high-level target while making the final click location inaccurate. Second, it can remove context that helps the model follow the correct action convention or distinguish between clicking an autocomplete suggestion and submitting an incomplete query. Future improvements should therefore protect both spatially important screenshot regions and textual action-history cues that define valid tool templates and task-specific interaction patterns.

5.14 Visualization of Multi-Stage Pruning

Stage 1 screenshot pruning. Stage 1 operates at the screenshot level. It evaluates the historical screenshots in the computer-use trajectory and removes low-importance screenshots before exact prefilling. In the Stage 1 visualizations, red screenshots indicate low-importance screenshots that are pruned. The purpose of this stage is to reduce temporal redundancy in the observation history while preserving screenshots that are more relevant to the current action. Figures 5.7, 5.8, and 5.9 on the following pages show examples of historical screenshots selected or removed by Stage 1.

Stage 2 token pruning. Stage 2 operates after Stage 1 and performs token-level compression on the retained reduced context. Different from Stage 1 screenshot pruning, Stage 2 works at the token or patch level. In the Stage 2 visualizations, red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. Figures 5.10, 5.11, and 5.12 on the following pages use the trajectory from Figure 5.9 to show how token-level pruning changes as the KV-cache budget becomes stricter or more relaxed.

All the visualization samples are generated using AgentNetBench dataset and Qwen3VL 8B model.

5.14. Visualization of Multi-Stage Pruning

Stage 1

User instruction: Compose and save a new email template titled 'Weekly Report' in Outlook, the content should include Hello at the beginning and name 'Xiaole' at last. Includes 4 points: Week, Hours of Work, Achievements, Money Spent.



FIGURE 5.7: Stage 1 example 1. Red screenshots indicate low-importance screenshots that are pruned. This example shows that visually similar past screenshots such as t-4, t-3 and t-2 are pruned.

Stage 1

User instruction: Open Microsoft Excel from the desktop. Create a new blank workbook. Enter 'title' in cell A1. Merge and center cells A1 to D1. Save the workbook as '1.xlsx' on the desktop.

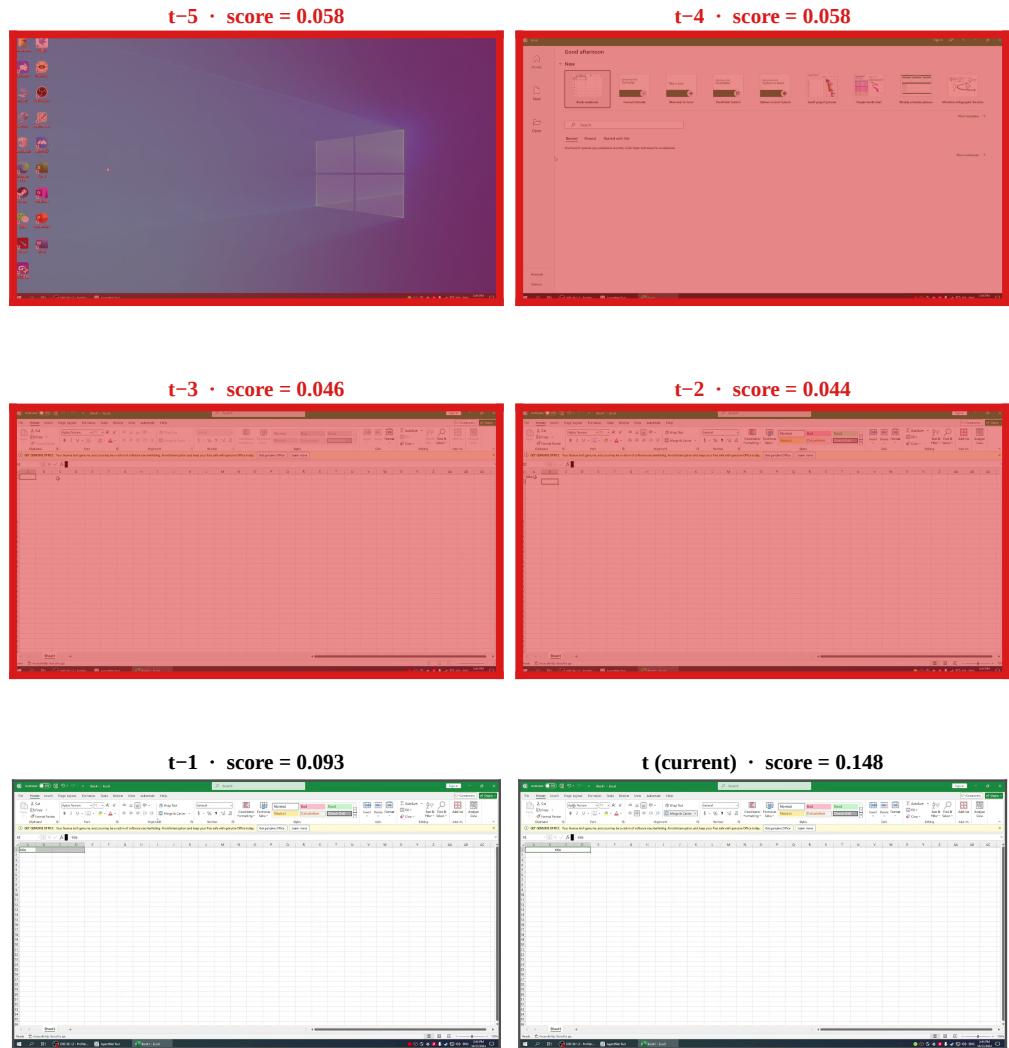


FIGURE 5.8: Stage 1 example 2. Red screenshots indicate low-importance screenshots that are pruned. In this example, t-5 and t-4 have the same low score, showing temporal redundancy, while the visually similar t-3 and t-2 are also pruned.

5.14. Visualization of Multi-Stage Pruning

Stage 1

User instruction: Open the 'split_csv_1' report in Microsoft Power BI. Export the report to Excel using the 'Analyze in Excel' option. Open the exported file in Excel for the web. Enable external data connections in Excel. Select 'instructions steps' and 'platform' fields in the PivotTable for analysis.

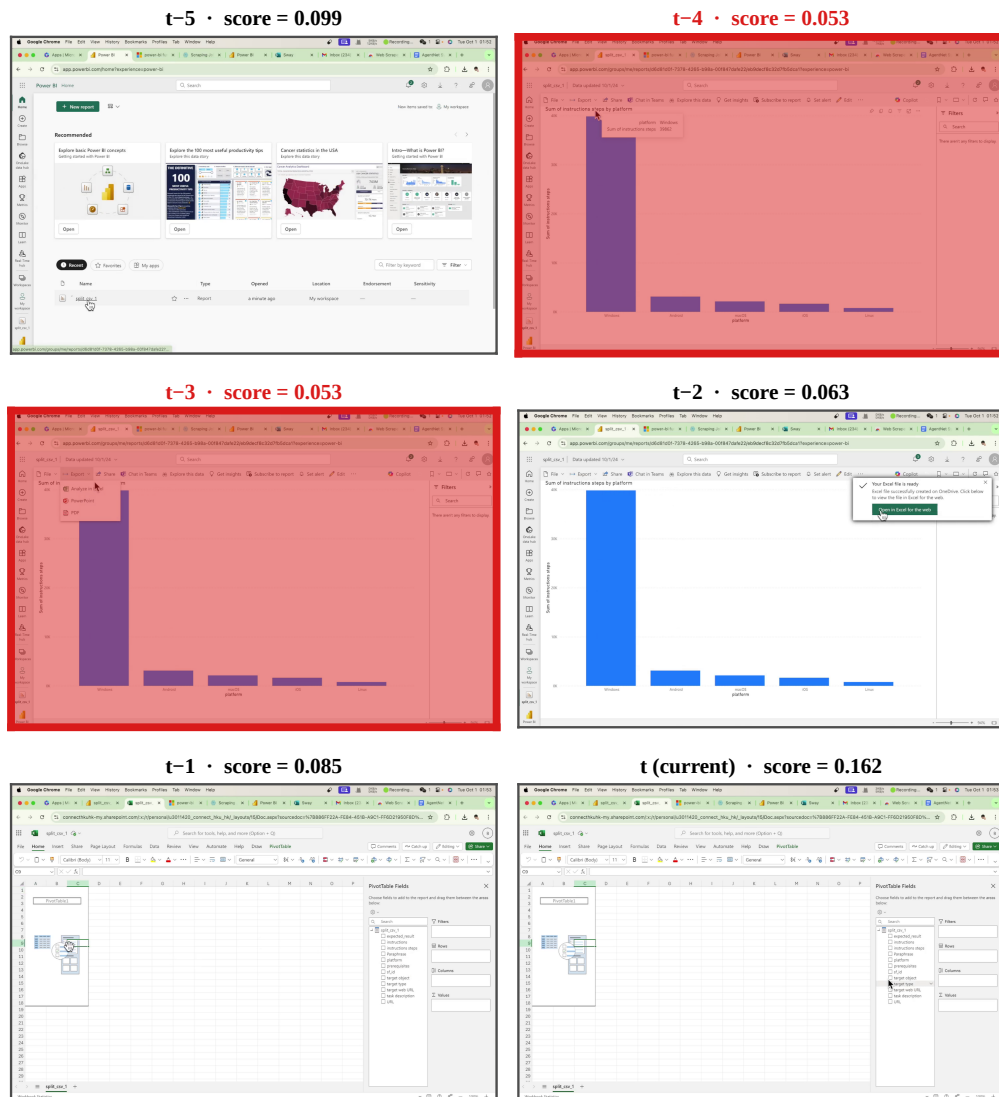


FIGURE 5.9: Stage 1 example 3. Red screenshots indicate low-importance screenshots that are pruned. This example shows that visually similar past screenshots such as t-4 and t-3 are pruned.

Stage 2 — target budget: 20%

User instruction: Open the 'split_csv_1' report in Microsoft Power BI. Export the report to Excel using the 'Analyze in Excel' option. Open the exported file in Excel for the web. Enable external data connections in Excel. Select 'instructions steps' and 'platform' fields in the PivotTable for analysis.



FIGURE 5.10: Stage 2 token pruning at the 20% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. This is the most aggressive setting, so only the most important tokens are directly retained.

5.14. Visualization of Multi-Stage Pruning

Stage 2 — target budget: 40%

User instruction: Open the 'split_csv_1' report in Microsoft Power BI. Export the report to Excel using the 'Analyze in Excel' option. Open the exported file in Excel for the web. Enable external data connections in Excel. Select 'instructions steps' and 'platform' fields in the PivotTable for analysis.



FIGURE 5.11: Stage 2 token pruning at the 40% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. Compared with the 20% case, more tokens are directly preserved.

Stage 2 — target budget: 60%

User instruction: Open the 'split_csv_1' report in Microsoft Power BI. Export the report to Excel using the 'Analyze in Excel' option. Open the exported file in Excel for the web. Enable external data connections in Excel. Select 'instructions steps' and 'platform' fields in the PivotTable for analysis.



FIGURE 5.12: Stage 2 token pruning at the 60% KV-cache budget. Red screenshots indicate low-importance screenshots that are pruned, while lighter red patches indicate low-importance tokens that will be compressed into a fixed-size summary state. This budget is more conservative, so a larger portion of the original context is retained.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis studied efficient context compression for long-horizon GUI agents. The central problem is that GUI trajectories create long multimodal prompts: historical screenshots contribute many visual tokens, while textual reasoning and action traces are needed to maintain task continuity. As the number of interaction steps grows, full-context inference becomes expensive in two ways. First, exact prefilling over the complete sequence incurs high attention cost. Second, decoding must repeatedly access a large KV cache. These costs are especially severe for GUI agents because many past screenshots are visually redundant or no longer relevant to the next action.

To address this problem, this thesis proposed a multi-stage KV-cache pruning framework that matches the structure of GUI trajectories. The first stage performs step-level pruning before exact prefill. Instead of pruning tokens from a flat sequence, it treats each historical screenshot as a structured visual block and estimates the effect of removing that block on the attention output. This design directly targets the dominant source of multimodal context length while preserving textual action history whenever possible. By removing less informative screenshot blocks before exact prefilling, the method reduces the sequence length processed by the expensive full-attention computation.

The thesis also introduced a Taylor-approximated scoring method for Stage 1 pruning. Exact softmax-based perturbation scoring provides a meaningful measure of screenshot-block importance, but computing it directly for every candidate block is too costly. The Taylor approximation replaces the expensive exact scoring process with a practical surrogate based on first-order expansion and reusable cumulative statistics. This keeps the pruning criterion aligned with the exact perturbation objective while making step scoring feasible for long GUI trajectories.

After step-level reduction, the second stage applies token-level pruning and hybrid decoding on the reduced KV cache. Important tokens are retained in a keep set and decoded with exact attention, while lower-importance tokens are represented through a compact summary contribution. This stage further reduces decode-time memory and computation without relying only on coarse step removal. Together, the two stages address complementary parts of the inference bottleneck: Stage 1 lowers prefill cost by shortening the input sequence before exact processing, and Stage 2 lowers decode-time cost by compressing the retained cache.

The experimental results support the value of structure-aware pruning for GUI agents. The transferred benchmark results show that historical visual context is useful only up to a limited point and that performance saturates or slightly decreases when too many screenshots are retained. This observation supports the motivation for removing redundant historical visual states. The hyperparameter study further shows that the proposed method can maintain stable performance across different KV-cache budgets when the scoring window and eviction threshold are chosen appropriately. Overall, the results indicate that GUI-agent compression should not treat all tokens uniformly; it should exploit

the natural separation between screenshot blocks and textual action history.

In summary, this thesis shows that KV-cache compression for GUI agents should be structure-aware. By pruning redundant screenshot blocks before exact prefill and then compressing token-level redundancy during decoding, the proposed framework offers a more suitable alternative to token-only pruning for screenshot-heavy long-horizon interaction.

6.2 Future Work

The following directions can further improve the proposed framework:

- **Lower-cost scoring.** The Taylor-based score reduces the cost of exact scoring, but it still adds overhead before prefill. Future work can make this step cheaper by scoring fewer screenshot blocks, reusing scores from previous steps, or using a simple coarse filter before applying the Taylor score.
- **Adaptive pruning budgets.** The current method uses fixed pruning settings. In future work, the amount of pruning can be adjusted based on task difficulty, model confidence, trajectory length, or the amount of visual change between screenshots.
- **Broader long-context applications.** The multi-stage design is inspired by the episodic structure of GUI agent inputs, where the model receives a sequence of images, actions, and reasoning history. Although this thesis focuses on GUI tasks, the same idea may be useful for other settings with long multimodal or conversational context. In robotics and embodied AI, agents observe streams of camera frames, sensor readings, instructions, and previous actions, so pruning redundant observations could reduce

inference cost while preserving task-relevant states such as object locations or recent action outcomes. In multi-turn chatbots, context pruning could preserve important user preferences, task constraints, and recent dialogue turns while removing redundant or less relevant conversation history.

Overall, future work should make the pruning process cheaper, more adaptive, and more closely connected to the actions required by GUI tasks, while also exploring whether structure-aware compression can benefit other long-context interactive systems.

Bibliography

- [1] S. Bai, Y. Cai, R. Chen, K. Chen *et al.*, “Qwen3-VL Technical Report,” *arXiv preprint arXiv:2511.21631*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.21631>
- [2] T. Xue, C. Peng, M. Huang, L. Guo *et al.*, “EvoCUA: Evolving Computer Use Agents via Learning from Scalable Synthetic Experience,” *arXiv preprint arXiv:2601.15876*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.15876>
- [3] Y. Qin, Y. Ye, J. Fang, H. Wang *et al.*, “UI-TARS: Pioneering Automated GUI Interaction with Native Agents,” *arXiv preprint arXiv:2501.12326*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.12326>
- [4] H. Wang, H. Zou, H. Song, J. Feng *et al.*, “UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning,” *arXiv preprint arXiv:2509.02544*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.02544>
- [5] K. Q. Lin, L. Li, D. Gao, Z. Yang *et al.*, “ShowUI: One Vision-Language-Action Model for GUI Visual Agent,” in *Proc. the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [6] X. Wang, B. Wang, D. Lu, J. Yang *et al.*, “OpenCUA: Open Foundations for Computer-Use Agents,” in *Proc. the Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit *et al.*, “Attention Is All You Need,” in *Proc. the Conference on Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [8] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [9] H. Touvron, L. Martin, K. Stone, P. Albert *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [10] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford *et al.*, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [11] A. Yang, A. Li, B. Yang, B. Zhang *et al.*, “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [12] Y. Feng, J. Lv, Y. Cao, X. Xie *et al.*, “Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference,” in *Proc. the Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [13] G. Xiao, Y. Tian, B. Chen, S. Han *et al.*, “Efficient Streaming Language Models with Attention Sinks,” in *Proc. the International Conference on Learning Representations (ICLR)*, 2024.
- [14] Z. Liu, A. Desai, F. Liao, W. Wang *et al.*, “Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at

BIBLIOGRAPHY

- Test Time,” in *Proc. the Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair *et al.*, “Keyformer: KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference,” in *Proc. Machine Learning and Systems (MLSys)*, 2024.
- [16] Y. Li, Y. Huang, B. Yang, B. Venkitesh *et al.*, “SnapKV: LLM Knows What You are Looking for Before Generation,” in *Proc. the Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [17] J. Tang, Y. Zhao, K. Zhu, G. Xiao *et al.*, “QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference,” in *Proc. the International Conference on Machine Learning (ICML)*, 2024.
- [18] Z. Cai, Y. Zhang, B. Gao, Y. Liu *et al.*, “PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling,” in *Proc. the Conference on Language Modeling (COLM)*, 2025.
- [19] Y. Fu, Z. Cai, A. Asi, W. Xiong *et al.*, “Not All Heads Matter: A Head-Level KV Cache Compression Method with Integrated Retrieval and Reasoning,” in *Proc. the International Conference on Learning Representations (ICLR)*, 2025.
- [20] H. Tang, Y. Lin, J. Lin, Q. Han *et al.*, “RazorAttention: Efficient KV Cache Compression Through Retrieval Heads,” in *Proc. the International Conference on Learning Representations (ICLR)*, 2025.
- [21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang *et al.*, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” in *Proc. the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

- [22] H. Jiang, Q. Wu, X. Luo, D. Li *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” in *Proc. the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [23] Z. Pan, Q. Wu, H. Jiang, M. Xia *et al.*, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” in *Findings of the Association for Computational Linguistics*, 2024.
- [24] L. Chen, H. Zhao, T. Liu, S. Bai *et al.*, “An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models,” in *Proc. the European Conference on Computer Vision (ECCV)*, 2024.
- [25] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng *et al.*, “SparseVLM: Visual Token Sparsification for Efficient Vision-Language Model Inference,” in *Proc. the International Conference on Machine Learning (ICML)*, 2025.
- [26] S. Yang, Y. Chen, Z. Tian, C. Wang *et al.*, “VisionZip: Longer is Better but Not Necessary in Vision Language Models,” in *Proc. the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [27] Z. Wan, Z. Wu, C. Liu, J. Huang *et al.*, “LOOK-M: Look-Once Optimization in KV Cache for Efficient Multimodal Long-Context Inference,” in *Findings of the Association for Computational Linguistics*, 2024.
- [28] K. Li, Z. Jiang, Z. Shen, Z. Wang *et al.*, “MadaKV: Adaptive Modality-Perception KV Cache Eviction for Efficient Multimodal Long-Context Inference,” in *Proc. the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

BIBLIOGRAPHY

- [29] M. Ouyang, K. Q. Lin, M. Z. Shou, and H. T. Ng, "FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection," in *Proc. the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [30] K.-H. Huang, H. Qiu, Y. Dai, C. Xiong *et al.*, "GUI-KV: Efficient GUI Agents via KV Cache with Spatio-Temporal Awareness," *arXiv preprint arXiv:2510.00536*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.00536>
- [31] T. Xie, D. Zhang, J. Chen, X. Li *et al.*, "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," in *Proc. the Conference on Neural Information Processing Systems (NeurIPS), Track on Datasets and Benchmarks*, 2024.
- [32] R. Niu, J. Li, S. Wang, Y. Fu *et al.*, "ScreenAgent: A Vision Language Model-driven Computer Control Agent," in *Proc. the International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [33] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz *et al.*, "AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents," in *Proc. the International Conference on Learning Representations (ICLR)*, 2025.

국문초록

GUI 에이전트를 위한 단계적 문맥 가지치기 방법

세종대학교 일반대학원

AI로봇학과

Nguyen Quoc Viet

그래픽 사용자 인터페이스(GUI) 에이전트는 스크린샷을 관찰하고 클릭, 입력, 스크롤과 같은 동작을 선택함으로써 사용자의 지시를 수행한다. 이러한 에이전트는 일반적으로 스크린샷에서 얻은 시각 토큰과 사용자 지시문의 텍스트 토큰을 결합하여 다음 동작을 예측하는 트랜스포머 아키텍처를 기반으로 한다. 주요 효율성 문제는 단일 동작만으로 끝나는 작업이 아니라, 목표 달성을 위해 일련의 GUI 조작이 필요한 다단계 작업에서 발생한다. 이러한 작업에서는 모델이 이전 스크린샷과 과거 동작을 기억해야 하므로 긴 멀티모달 문맥이 생성되며, 이로 인해 프리필링 단계와 키-값 캐시 디코딩 단계 모두에서 추론 비용이 증가한다. 본 논문은 GUI 에이전트 추론의 효율성 문제를 해결하기 위해 다단계 문맥 가지치기 프레임워크를 제안한다. 제안 방법은 먼저 테일러 기반 중요도 점수를 활용하여 본격적인 프리필링을 수행하기 전에 상대적으로 중요도가 낮은 과거 스크린샷 블록을 제거한다. 이후 토큰 수준의 가지치기와 하이브리드 디코딩을 적용하여,

동작 예측에 필요한 과거 상호작용 정보를 보존하면서도 디코딩 시점의 키-값 캐시 비용을 줄인다. AgentNetBench와 OSWorld에서 수행한 실험 결과, 제안 프레임워크는 압축된 키-값 캐시에서도 성능을 유지함을 보였다. AgentNetBench에서 키-값 캐시를 20%만 유지한 경우에도 Qwen3VL 8B는 78.3%, EvoCUA 8B는 57.2%의 단계 성공률을 달성하였으며, 이는 강한 압축 조건에서도 행동 품질이 상당히 유지됨을 보여준다. 또한 캐시 유지 비율을 40%와 60%로 증가시켰을 때, Qwen3VL 8B는 각각 81.2%와 82.0%, EvoCUA 8B는 각각 63.6%와 64.8%의 단계 성공률을 달성하였으며, 이는 전체 캐시를 사용하는 경우와 동등하거나 약간 더 높은 성능이다. 이러한 결과는 GUI 에이전트의 문맥을 단순한 평면적 토큰 시퀀스로 취급하기보다, 상호작용 구조에 따라 압축해야 함을 시사한다.

키워드: 문맥 가지치기, GUI 에이전트, 키-값 캐시, 시각-언어 모델.

Acknowledgement

First of all, I would like to express my deepest gratitude to my supervisor, Professor Cheol Jeong. His guidance and advice have been essential throughout my master's study. Through his supervision, I learned how to approach research problems more carefully, think more critically, and improve my work step by step. I am especially grateful for his patience and support, which helped me continue my research and complete both this thesis and my research paper.

I would also like to give my sincere thanks to my committee members for their valuable comments, and time. Their feedback helped me improve the quality of this thesis and encouraged me to think more clearly about the contribution and direction of my research.

I am deeply grateful to all members of VLILAB for creating a supportive and inspiring research environment. The discussions, shared experiences, and daily interactions in the lab helped me grow not only as a researcher, but also as a person. I truly appreciate the help, encouragement, and friendship I received from the lab members throughout this journey.

In particular, I would like to give special thanks to Thang and Qiang. Their generous support, thoughtful advice, and willingness to share their experience helped me greatly during many stages of my research. I am thankful for the discussions we had about ideas, experiments, writing, and presentation. Their patience and encouragement made difficult tasks feel more manageable and gave me confidence to keep improving my work.

I am also grateful to my friends, who shared both joy and sorrow with me during my time in Korea. Their companionship, kindness, and encouragement

gave me strength and motivation during stressful periods. The memories we created together made my graduate life more meaningful and unforgettable.

Finally, I would like to thank my family from the bottom of my heart. Your unconditional love, trust, and support are everything to me. Even from far away, your belief in me has always been my greatest source of strength. I will carry this love and trust with me, continue moving forward, and always try my best to make you proud.